东北大学 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY



# Python for Data Analysis
## —— PART II Improved

1


雨课堂 Rain Classroom

# Plotting and Visualization

## Matplotlib

- Making informative visualizations (sometimes called ***plots***) is one of the most important tasks in data analysis.
- It may be a part of the exploratory process—for example, identify outliers(识别异常值) or data transformations, or generating ideas for models.
- matplotlib is a desktop plotting package designed for creating (mostly two-dimensional) publication-quality plots.
- matplotlib supports various GUI backends on all operating systems and additionally can export visualizations to all of the common formats (PDF, SVG, JPG, PNG, BMP, GIF, etc.)
- Matplotlib——MATLAB+plot(绘图)+library(集合)

# Plotting and Visualization

## pyplot

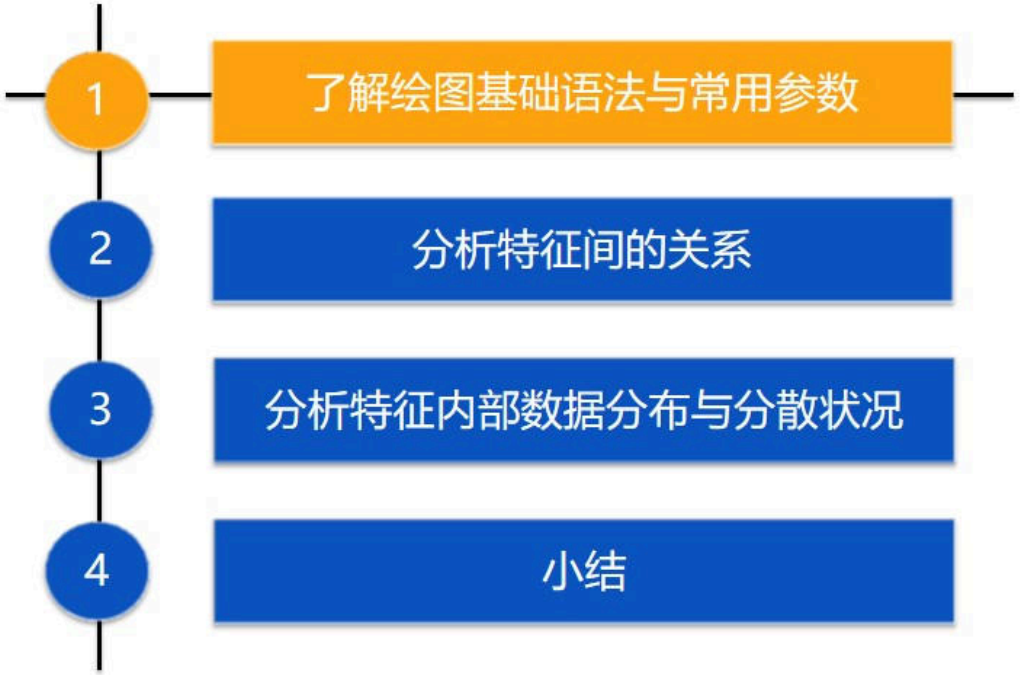
- matplotlib.pyplot (以下简称pyplot)是一个命令风格函数的集合，使Matplotib的机制更像MATLAB。
- 每个绘图函数都可对图形进行一些更改，如创建图形，在图形中创建绘图区域，在绘图区域绘制-些线条，使用标签装饰绘图等。
- 在pyplot中，各种状态跨函数调用保存，以便跟踪诸如当前图形和绘图区域之类的东西，并且绘图函数始终指向当前轴域。
- 本章将以pyplot为基础，讲述5种基础统计图形的绘制方法。
- 参考：https://matplotlib.org/stable/





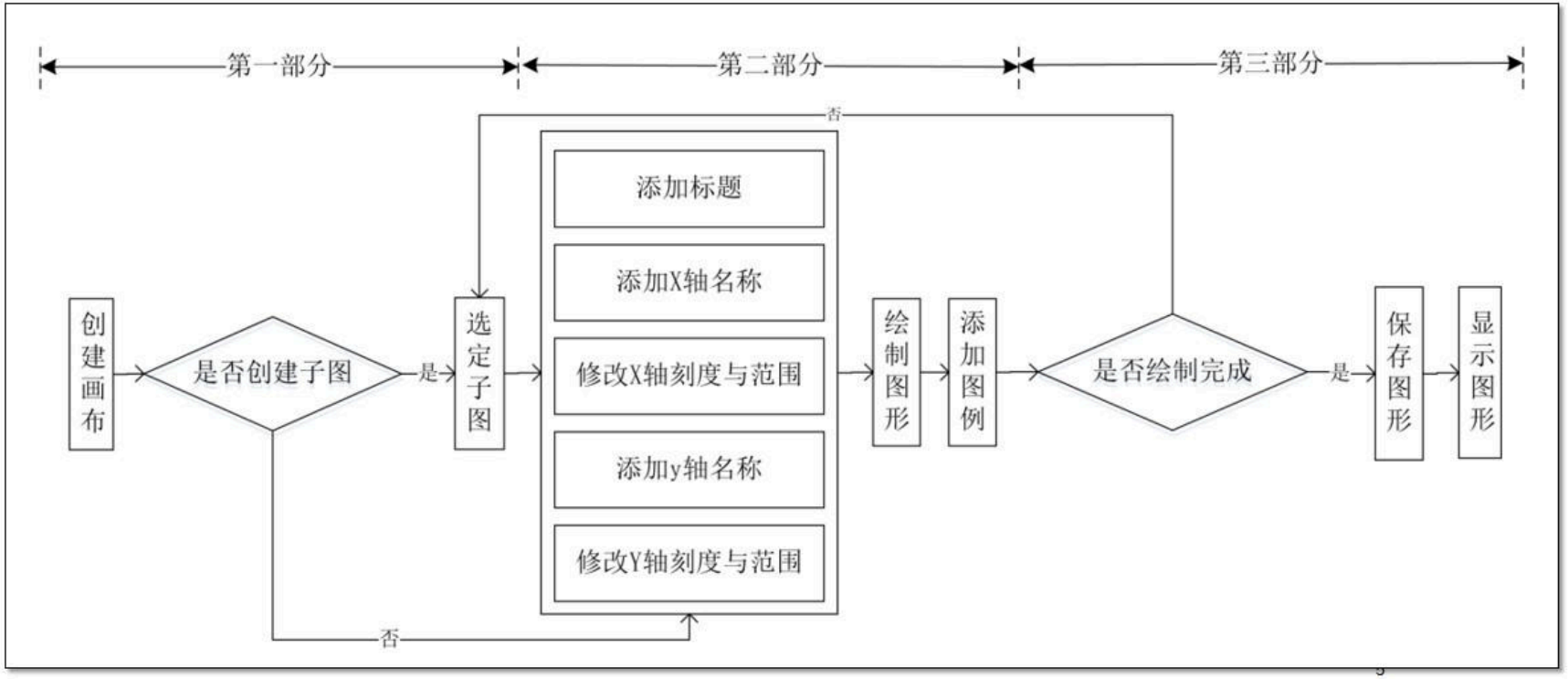
雨课堂 Rain Classroom

# 目录



# A Brief matplotlib API Primer

## Basic plotting Flow



# A Brief matplotlib API Primer

## 1. Figures and Subplots(创建画布，绘制子图)

- Plots in matplotlib reside within a Figure object. You can create a new figure with *plt.figure*: `fig = plt.figure()`
- You can't make a plot with a blank figure. You have to create one or more subplots using *add_subplot*: `ax1 = fig.add_subplot(2, 2, 1)`

| methods | Description |
| --- | --- |
| plt.figure | 创建一个空白画布（返回画布对象），可以指定画布大小，像素。 |
| figure.add_subplot | 创建并选中子图，可以指定子图的行数，列数，与选中图片编号。 |

6


雨课堂 Rain Classroom

# A Brief matplotlib API Primer

## 2. Ticks, Labels, and Legends(添加画布内容)

- Add plot decorations

| methods | Description |
| --- | --- |
| plt.title | 在当前图形中添加标题，可以指定标题的名称、位置、颜色、字体大小等参数。 |
| plt.xlabel | 在当前图形中添加x轴名称，可以指定位置、颜色、字体大小等参数。 |
| plt.ylabel | 在当前图形中添加y轴名称，可以指定位置、颜色、字体大小等参数。 |
| plt.xlim | 指定当前图形x轴的范围，只能确定一个数值区间，而无法使用字符串标识。 |
| plt.ylim | 指定当前图形y轴的范围，只能确定一个数值区间，而无法使用字符串标识。 |
| plt.xticks | 指定x轴刻度的数目与取值。 |
| plt.yticks | 指定y轴刻度的数目与取值。 |
| plt.legend | 指定当前图形的图例，可以指定图例的大小、位置、标签。 |


雨课堂 Rain Classroom

# A Brief matplotlib API Primer

## 3. Saving Plots to File

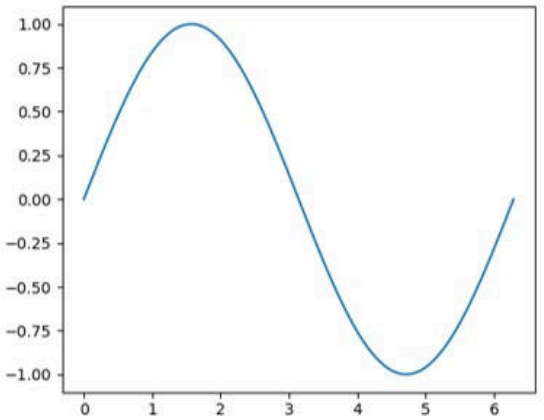
| methods | Description |
| --- | --- |
| plt.savafig | 保存绘制的图片，可以指定图片的分辨率、边缘的颜色等参数。 |
| plt.show | 在本机显示图形。 |

# A Brief matplotlib API Primer

## Draw a first plot

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2*np.pi, 200)
y = np.sin(x)
```



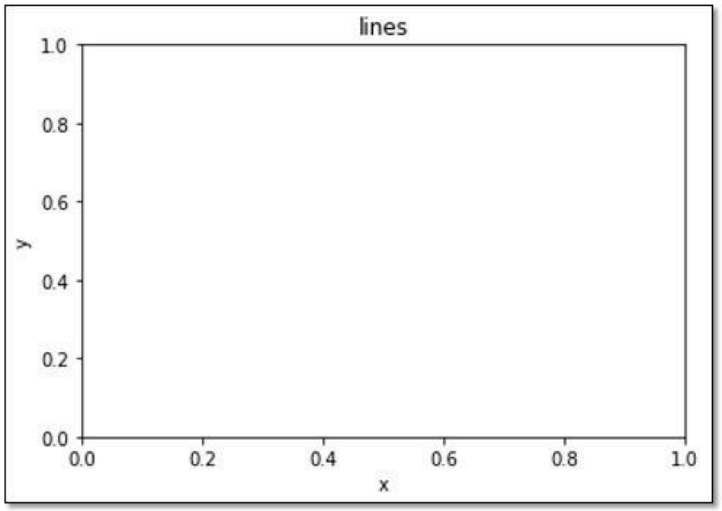

```
fig, ax = plt.subplots()        # Create a figure containing a single axes.
ax.plot(x, y)                   # Plot some data on the axes.
plt.show()
```

# A Brief matplotlib API Primer

**Example:**

```
import numpy as np
import matplotlib.pyplot as plt

plt.title('lines')      # 添加标题
plt.xlabel('x')         # 添加x轴的名称
plt.ylabel('y')         # 添加y轴的名称
plt.xlim((0,1))         # 确定x轴范围
plt.ylim((0,1))         # 确定y轴范围
```



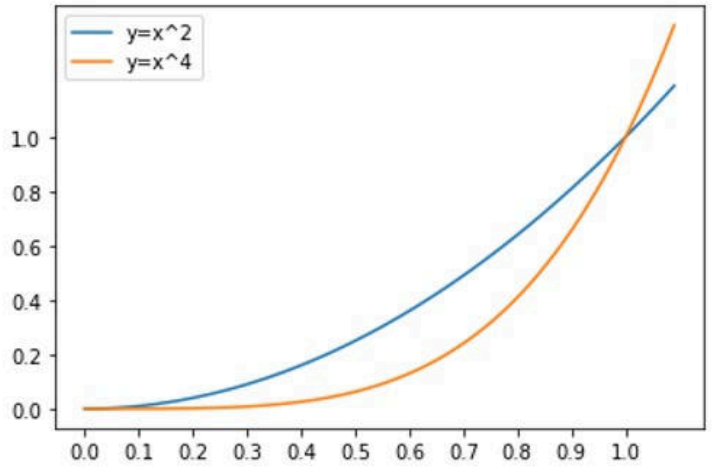

■ 省略了第一步（创建空白画布），直接在默认的画布上进行图形绘制

# A Brief matplotlib API Primer

**Example:**

```
plt.xticks(np.arange(0, 1.1, 0.1))      # 规定x轴刻度
plt.yticks([0,0.2,0.4,0.6,0.8,1])       # 确定y轴刻度
data = np.arange(0, 1.01, 0.01)         # 生成100个值
plt.plot(data, data**2)                 # 添加y=x^2曲线
plt.plot(data, data**4)                 # 添加y=x^4曲线
plt.legend(['y=x^2','y=x^4'])           #添加图例

plt.savefig('y=x^2.png')                #保存图片
plt.show()
```







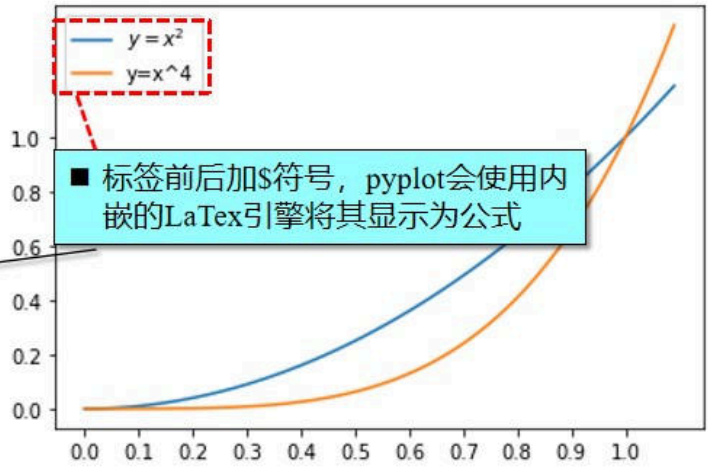

雨课堂 Rain Classroom

# A Brief matplotlib API Primer

**Example:**

```
plt.xticks(np.arange(0, 1.1, 0.1))      # 规定x轴刻度
plt.yticks([0,0.2,0.4,0.6,0.8,1])       # 确定y轴刻度
data = np.arange(0, 1.01, 0.01)         # 生成100个值
plt.plot(data, data**2)                 # 添加y=x^2曲线
plt.plot(data, data**4)                 # 添加y=x^4曲线
plt.legend(['$y=x^2$','y=x^4'])         #添加图例

plt.savefig('y=x^2.png')                #保存图片
plt.show()
```

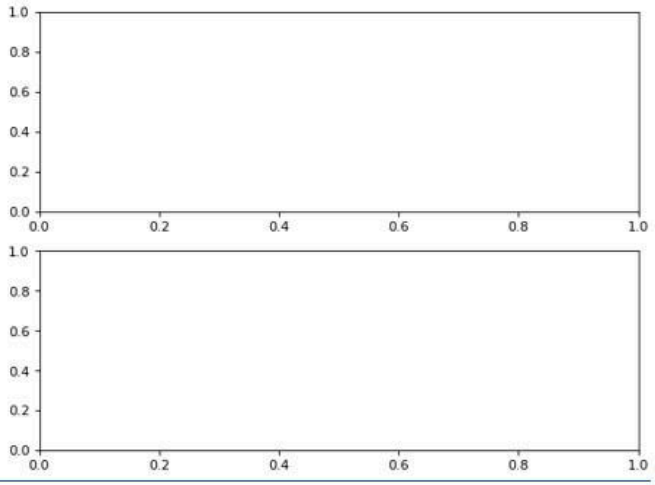



■ 标签前后加$符号，pyplot会使用内嵌的LaTex引擎将其显示为公式

# A Brief matplotlib API Primer

**Example： Subplots**

```
fig = plt.figure(figsize=(8,6),dpi=80)        # 确定画布大小
ax1 = fig.add_subplot(2,1,1)                  # 创建一个两行1列的子图1，返回axes对象
ax2 = fig.add_subplot(2,1,2)                  # 创建一个两行1列的子图2，返回axes对象
```




```
plt.sca(ax1)                  # 第一幅子图
plt.title('lines')            # 添加标题
plt.xlabel('x')               # 添加x轴的名称
plt.ylabel('y')               # 添加y轴的名称
plt.xlim((0,1))               # 确定x轴范围
plt.ylim((0,1))               # 确定y轴范围
```





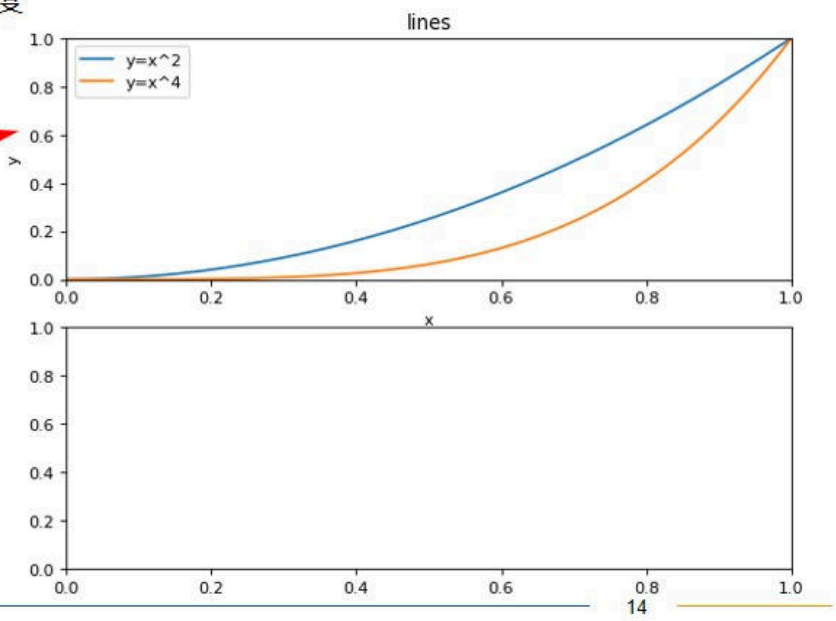
雨课堂 Rain Classroom

# A Brief matplotlib API Primer

**Example： Subplots**

```
plt.xticks([0,0.2,0.4,0.6,0.8,1])        # 规定x轴刻度
plt.yticks([0,0.2,0.4,0.6,0.8,1])        # 确定y轴刻度

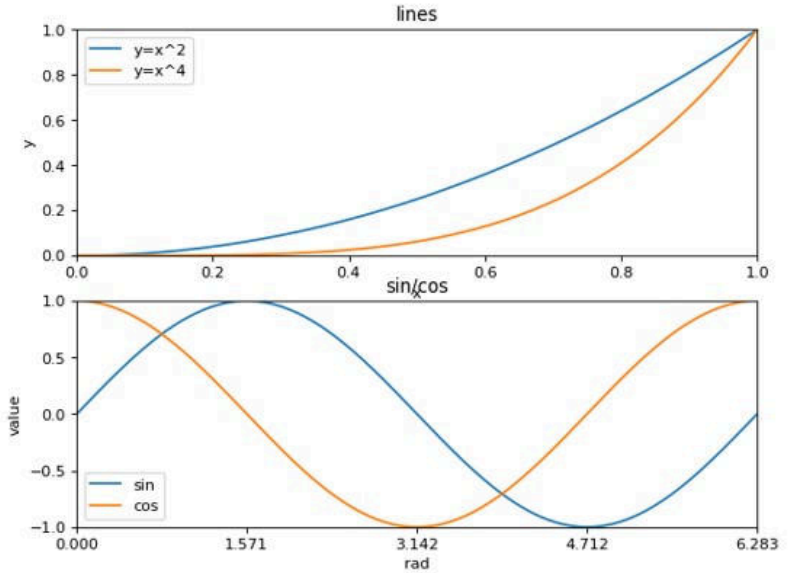

rad = np.arange(0, np.pi*2, 0.01)
plt.plot(rad, rad**2)                    # 添加y=x^2曲线
plt.plot(rad, rad**4)                    # 添加y=x^4曲线
plt.legend(['y=x^2','y=x^4'])
```



# A Brief matplotlib API Primer

## Example： Subplots

```
plt.sca(ax2)                    # 开始绘制第2幅
plt.title('sin/cos')            # 添加标题
plt.xlabel('rad')               # 添加x轴的名称
plt.ylabel('value')             # 添加y轴的名称
plt.xlim((0, np.pi*2))          # 确定x轴范围
plt.ylim((-1, 1))               # 确定y轴范围
plt.xticks([0, np.pi/2, np.pi, np.pi*1.5, np.pi*2])
plt.yticks([-1, -0.5, 0, 0.5, 1]) # 确定y轴刻度
plt.plot(rad, np.sin(rad))      # 添加sin曲线
plt.plot(rad, np.cos(rad))      # 添加cos曲线
plt.legend(['sin', 'cos'])
plt.savefig('sincos.png')
plt.show()
```



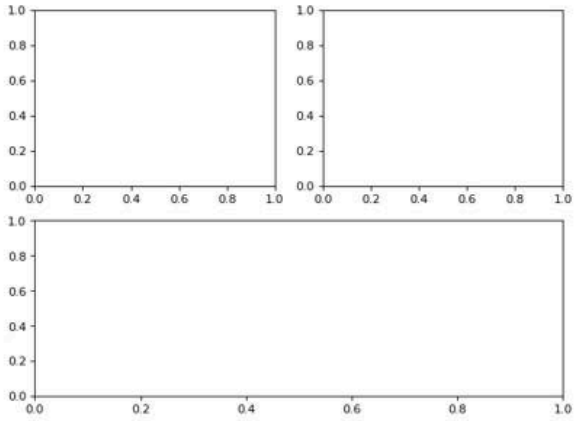





雨课堂 Rain Classroom

# A Brief matplotlib API Primer

**Example： Subplots**



```
import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(8,6), dpi=80)          # 确定画布大小
ax1 = fig.add_subplot(2,2,1)                     # 创建一个两行2列的第1行子图1，返回axes对象
ax2 = fig.add_subplot(2,2,2)                     # 创建一个两行2列的第1行子图2，返回axes对象
ax3 = fig.add_subplot(2,1,2)                     # 创建一个两行1列的第2行子图，返回axes对象
```



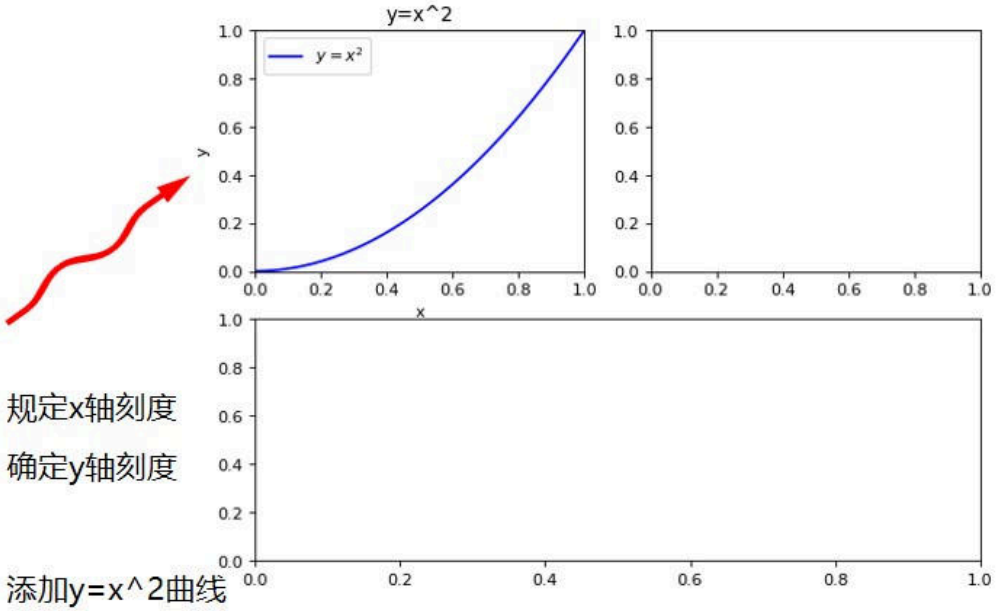


雨课堂 Rain Classroom

# A Brief matplotlib API Primer

## Example： Subplots

```
plt.sca(ax1)                                    # 第一幅子图
plt.title('y=x^2')                              # 添加标题
plt.xlabel('x')                                 # 添加x轴的名称
plt.ylabel('y')                                 # 添加y轴的名称
plt.xlim((0,1))                                 # 确定x轴范围
plt.ylim((0,1))                                 # 确定y轴范围
plt.xticks([0,0.2,0.4,0.6,0.8,1])               # 规定x轴刻度
plt.yticks([0,0.2,0.4,0.6,0.8,1])               # 确定y轴刻度
rad = np.arange(0, np.pi*2, 0.01)
plt.plot(rad, rad**2, color='blue')             # 添加y=x^2曲线
plt.legend(['$y=x^2$'])
```





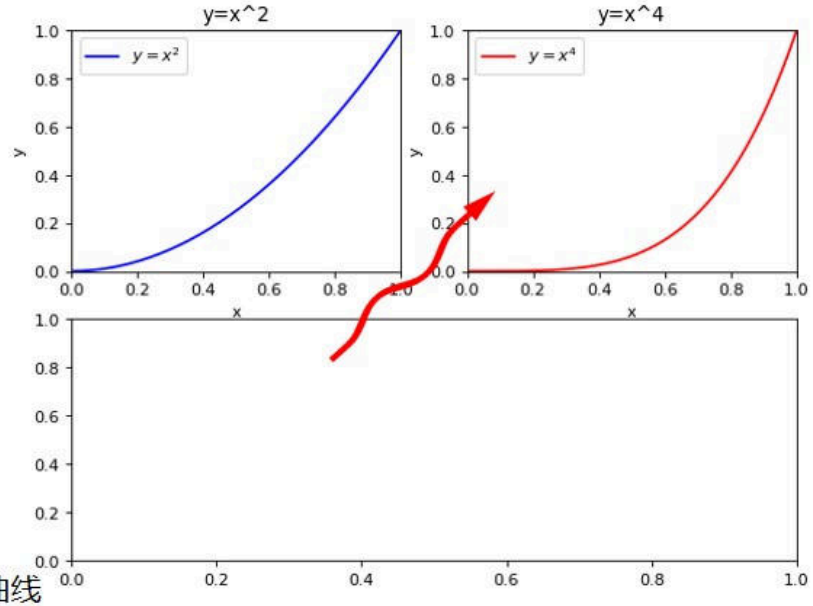



雨课堂 Rain Classroom

# A Brief matplotlib API Primer

## Example： Subplots

```
plt.sca(ax2)                                  # 第二幅子图
plt.title('y=x^4')                            # 添加标题
plt.xlabel('x')                               # 添加x轴的名称
plt.ylabel('y')                               # 添加y轴的名称
plt.xlim((0,1))                               # 确定x轴范围
plt.ylim((0,1))                               # 确定y轴范围
plt.xticks([0,0.2,0.4,0.6,0.8,1])             # 规定x轴刻度
plt.yticks([0,0.2,0.4,0.6,0.8,1])             # 确定y轴刻度
rad = np.arange(0, np.pi*2, 0.01)
plt.plot(rad, rad**4, color='red')            # 添加y=x^4曲线
plt.legend(['$y=x^4$'])
```

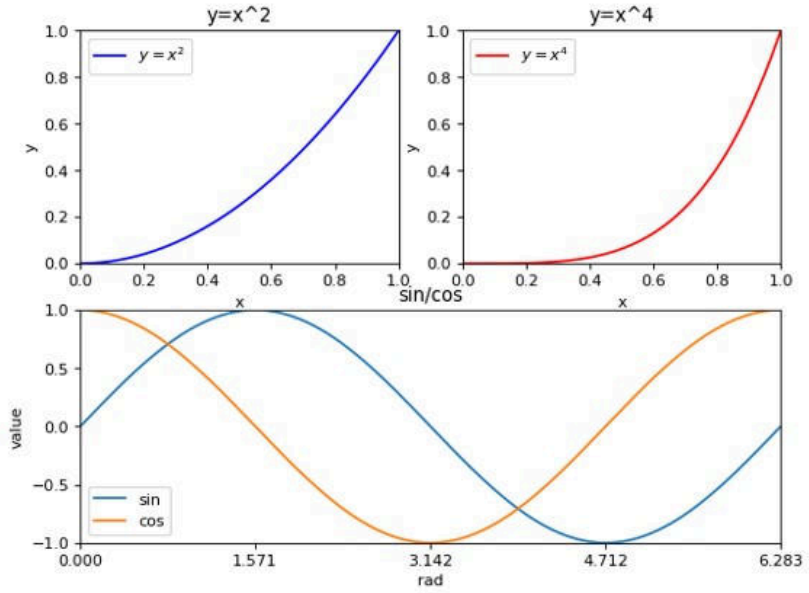


# A Brief matplotlib API Primer

**Example： Subplots**

```
plt.sca(ax3)                    # 开始绘制第3幅
plt.title('sin/cos')            # 添加标题
plt.xlabel('rad')               # 添加x轴的名称
plt.ylabel('value')             # 添加y轴的名称
plt.xlim((0, np.pi*2))          # 确定x轴范围
plt.ylim((-1, 1))               # 确定y轴范围
plt.xticks([0, np.pi/2, np.pi, np.pi*1.5, np.pi*2])
plt.yticks([-1, -0.5, 0, 0.5, 1]) # 确定y轴刻度
plt.plot(rad, np.sin(rad))      # 添加sin曲线
plt.plot(rad, np.cos(rad))      # 添加cos曲线
plt.legend(['sin','cos'])

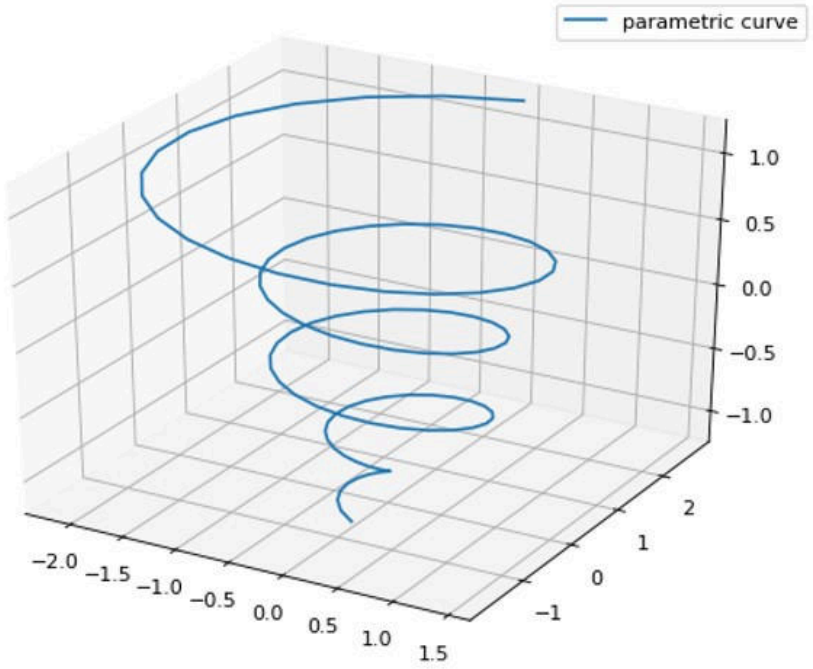
plt.savefig('sincos.png')
plt.show()
```



# A Brief matplotlib API Primer

## Example： Draw 3D images

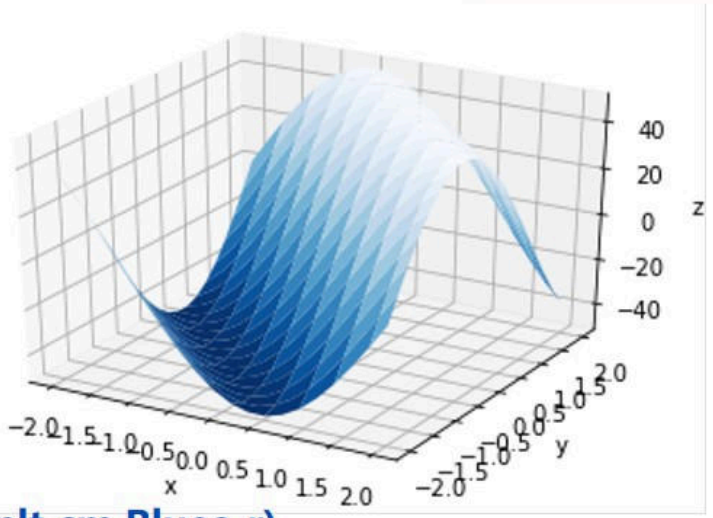
```
import matplotlib as mpl
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(8,6),dpi=80)      # 确定画布大小
ax = plt.subplot(projection='3d')           # 三维图形
theta = np.linspace(-4*np.pi, 4*np.pi, 100)
z = np.linspace(-4, 4, 100)*0.3
r = z**3+1
x = r * np.sin(theta)
y = r * np.cos(theta)
ax.plot(x,y,z, label='parametric curve')
ax.legend()
plt.show()
```



# A Brief matplotlib API Primer

**Example： Draw 3D images**

```
import numpy as np
import matplotlib.pyplot as plt
import mpl_toolkits.mplot3d
x, y =np.mgrid[-2:2:20j, -2:2:20j]      # 测试数据
z = 50 * np.sin(x+y)
ax = plt.subplot(111, projection='3d')
ax.plot_surface(x,y,z, rstride=2, cstride=1, cmap=plt.cm.Blues_r)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
plt.show()
```





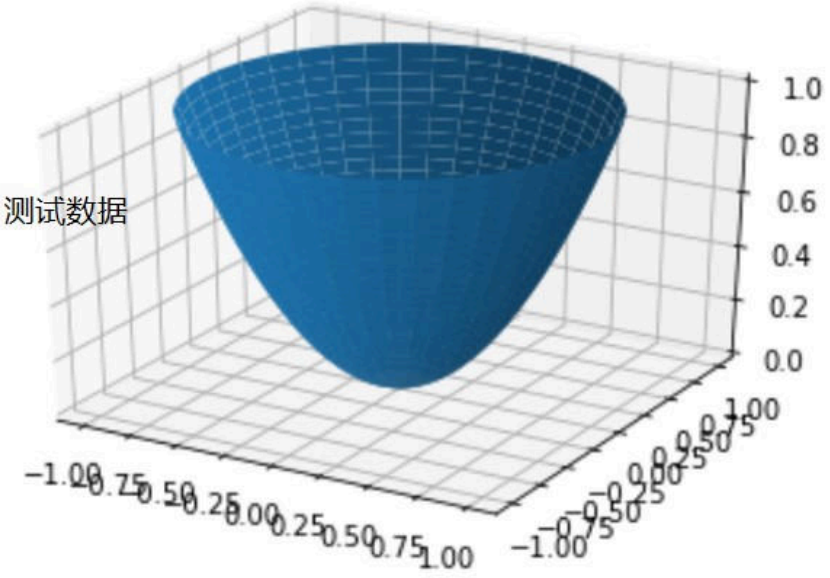



雨课堂 Rain Classroom

# A Brief matplotlib API Primer

**Example： Draw 3D images**

```
import numpy as np
import matplotlib.pyplot as plt
import mpl_toolkits.mplot3d
rho, theta =np.mgrid[0:1:40j, 0:2*np.pi:40j]  # 测试数据
z = rho ** 2
x = rho * np.cos(theta)
y = rho * np.sin(theta)
ax = plt.subplot(111, projection='3d')
ax.plot_surface(x,y,z)
plt.show()
```





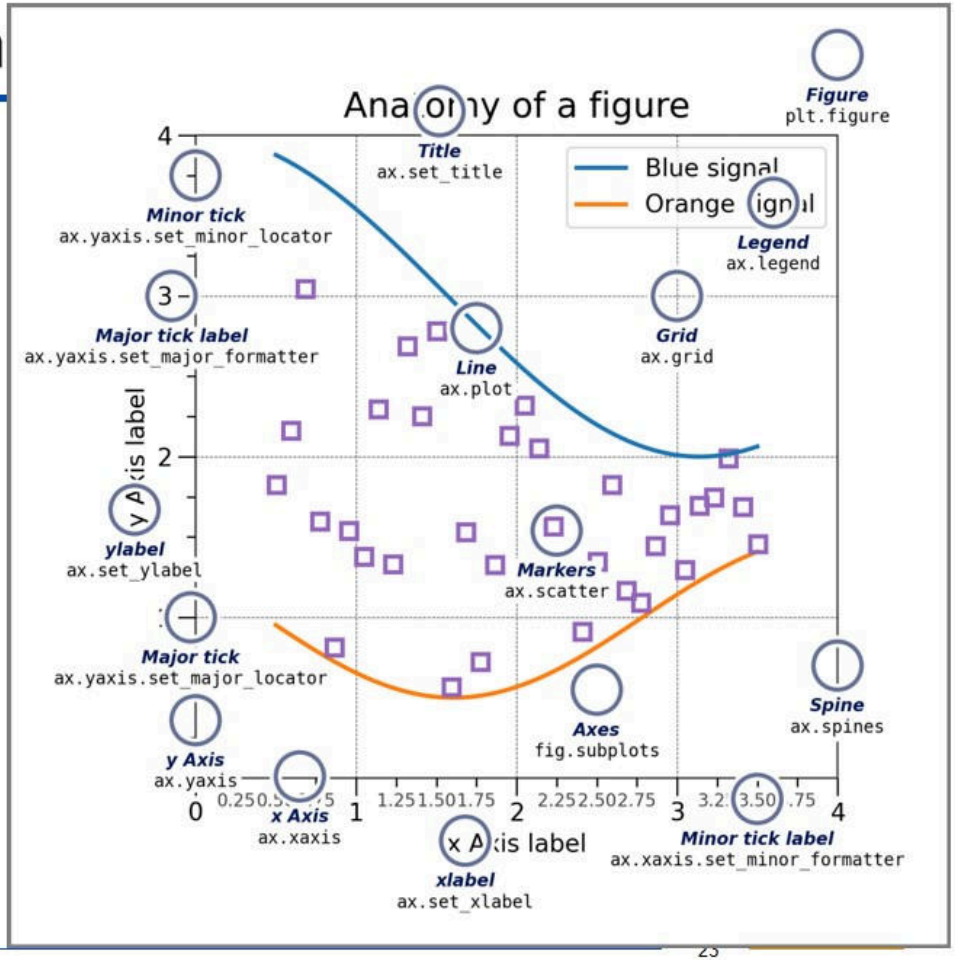



雨课堂 Rain Classroom

# A Brief matplotlib API Prim

## Parts of a Figure

- Here are the components of a Matplotlib Figure.
  - Figure
  - Axes
  - Title
  - xlabel，ylabel
  - Major tick label，Minor tick label
  - Major tick，Minor tick
  - Grid，Spine
  - x Axis，y Axis
  - Legend
  - Line，Markers

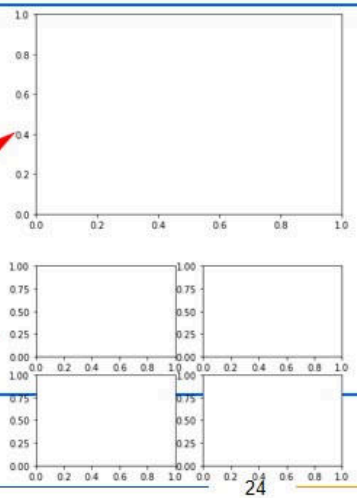


# A Brief matplotlib API Primer

## Figure

- The **whole** figure. The Figure keeps track of all the child Axes, a group of 'special' Artists (titles, figure legends, colorbars, etc), and even nested subfigures.
- The easiest way to create a new Figure is with pyplot:

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
fig = plt.figure()          # an empty figure with no Axes
                            # using fig.add_subplot()
fig, ax = plt.subplots()    # a figure with a single Axes
fig, axs = plt.subplots(2, 2)   # a figure with a 2x2 grid of Axes
```



24

# A Brief matplotlib API Primer

## Axes，Axis，Artist

- An **Axes**(子图) is an Artist attached to a Figure that contains a region for plotting data, and usually includes two (or three in the case of 3D) Axis objects.
- Axis(轴)：These objects set the scale and limits and generate ticks (the marks on the **Axis**) and ticklabels (strings labeling the ticks).
- Artist：Everything visible on the Figure is an **Artist** (even Figure, Axes, and Axis objects). When the Figure is rendered(渲染), all of the Artists are drawn to the canvas(画布).





雨课堂 Rain Classroom

# A Brief matplotlib API Primer

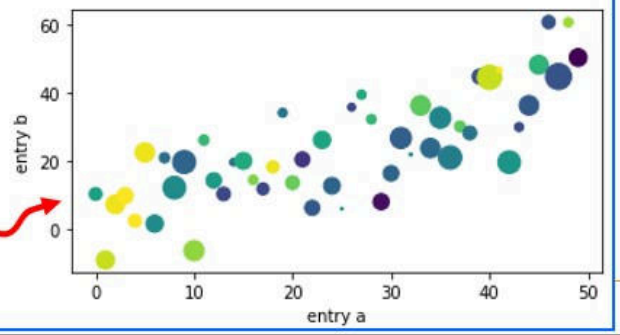
## Types of inputs to plotting functions

- Plotting functions expect ***numpy.array*** or ***numpy.ma.masked_array*** as input
- Most methods will also parse an ***addressable object***(可寻址对象) like a dict, a numpy.recarray, or a pandas.DataFrame.

```
np.random.seed(2022)    # seed the random number generator.
data = {'a': np.arange(50),
        'c': np.random.randint(0, 50, 50),
        'd': np.random.randn(50)}
data['b'] = data['a'] + 10 * np.random.randn(50)
data['d'] = np.abs(data['d']) * 100

fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.scatter('a', 'b', c='c', s='d', data=data)
ax.set_xlabel('entry a')
ax.set_ylabel('entry b');
```

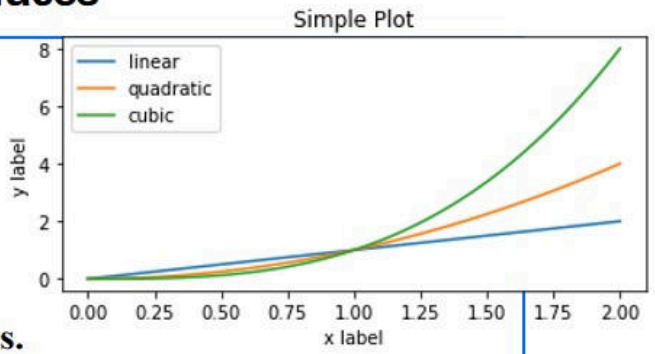
- 'a', 'b'位置参数，指定x, y的值，具体值从data数据中以key索引获取。
- c 参数是color列表，s参数是点的size列表，这里都是随机数



# A Brief matplotlib API Primer

## Coding styles : The object-oriented and the pyplot interfaces

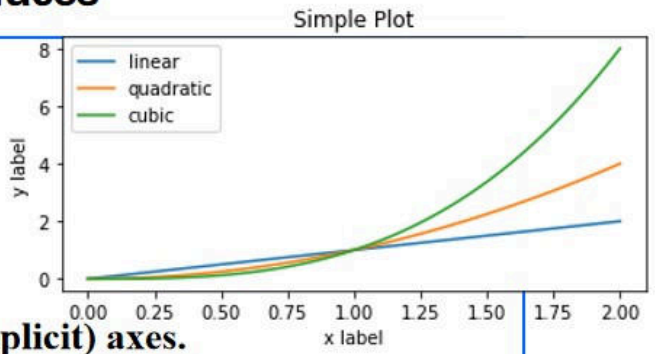
```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 2, 100)              # Sample data.
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
ax.plot(x, x, label='linear')           # Plot some data on the axes.
ax.plot(x, x**2, label='quadratic')     # Plot more data on the axes...
ax.plot(x, x**3, label='cubic')         # ... and some more.
ax.set_xlabel('x label')                # Add an x-label to the axes.
ax.set_ylabel('y label')                # Add a y-label to the axes.
ax.set_title("Simple Plot")             # Add a title to the axes.
ax.legend();                            # Add a legend.
```



# A Brief matplotlib API Primer

## Coding styles : The object-oriented and the pyplot interfaces

```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 2, 100)              # Sample data.
plt.figure(figsize=(5, 2.7), layout='constrained')
plt.plot(x, x, label='linear')          # Plot some data on the (implicit) axes.
plt.plot(x, x**2, label='quadratic')    # etc.
plt.plot(x, x**3, label='cubic')
plt.xlabel('x label')
plt.ylabel('y label')
plt.title("Simple Plot")
plt.legend();
```

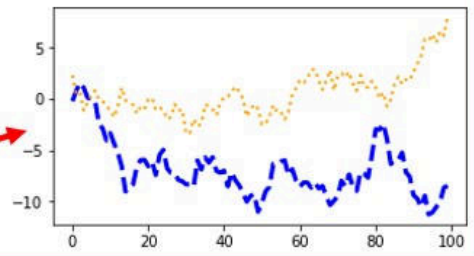


# A Brief matplotlib API Primer

## Styling Artists(配置绘图参数)

- Most plotting methods have styling options for the Artists, ①accessible either when a plotting method is called, or ②from a "setter" on the Artist.
- In the plot below we manually set the ***color***, ***linewidth***, and ***linestyle*** of the Artists created by plot, and we set the linestyle of the second line after the fact with set_linestyle.

```
data1, data2, data3, data4 = np.random.randn(4, 100)    # make 4 random data sets
fig, ax = plt.subplots(figsize=(5, 2.7))
x = np.arange(len(data1))
ax.plot(x, np.cumsum(data1), color='blue', linewidth=3, linestyle='--')
l, = ax.plot(x, np.cumsum(data2), color='orange', linewidth=2)
l.set_linestyle(':');
```

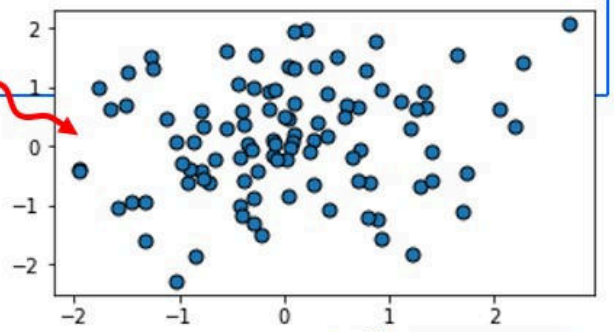


# A Brief matplotlib API Primer

## Colors

- Matplotlib has a very flexible array of colors that are accepted for most Artists; see the colors tutorial for a list of specifications.
- Some Artists will take multiple colors. i.e. for a scatter plot, the edge of the markers can be different colors from the interior:

```
data1, data2, data3, data4 = np.random.randn(4, 100)    # make 4 random data sets
fig, ax = plt.subplots(figsize=(5, 2.7))
ax.scatter(data1, data2, s=50, facecolor='C0', edgecolor='k');
```



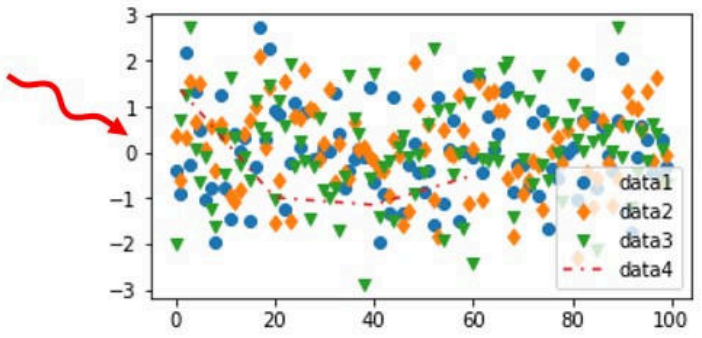
# A Brief matplotlib API Primer

## Linewidths, linestyles, and markersizes

- ***Line widths*** are typically in typographic points(印刷点1pt=1/72 inch) and available for Artists that have stroked lines(描边).
- Stroked lines can have a ***linestyle***.
- ***Marker size*** depends on the method being used (.plot通常是直径，.scatter通常是marker在区域内的比例)

```
data1, data2, data3, data4 = np.random.randn(4, 100)
fig, ax = plt.subplots(figsize=(5, 2.7))
ax.plot(data1, 'o', label='data1')
ax.plot(data2, 'd', label='data2')
ax.plot(data3, 'v', label='data3')
l, = ax.plot([1,20,40,60], data4[0:4], label='data4')
l.set_linestyle((0, (3, 3, 1, 3)))
ax.legend()
```

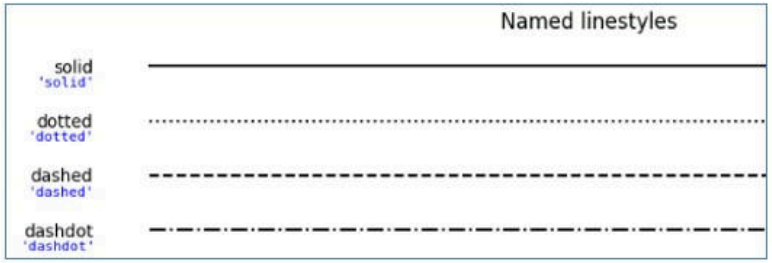
*# make 4 random data sets*



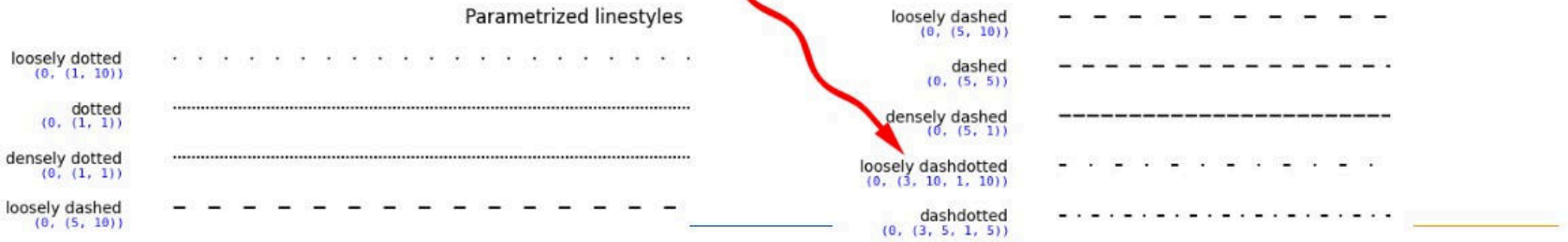
# A Brief matplotlib API Primer

## Linestyles

- Simple linestyles can be defined using the strings "*solid*", "*dotted*", "*dashed*" or "*dashdot*".




- More refined control can be achieved by providing a dash tuple ***(offset, (on_off_seq))***.
  - For example, (0, (3, 10, 1, 10)) means (3pt line, 10pt space, 1pt line, 10pt space) with no offset.



# A Brief matplotlib API Primer

## markers

- Functions to handle markers; used by the marker functionality of plot, scatter, and errorbar.

| marker | symbol | description | marker | symbol | description | marker | symbol | description |
|---|---|---|---|---|---|---|---|---|
| "." | | point | "s" | | square | "_" | | hline |
| "," | | pixel | "p" | | pentagon | 0(TICKLEFT) | | tickleft |
| "o" | | circle | "P" | | plus (filled) | 1(TICKRIGHT) | | tickright |
| "v" | | triangle_down | "*" | | star | 2(TICKUP) | | tickup |
| "^" | | triangle_up | "h" | | hexagon1 | 3(TICKDOWN) | | tickdown |
| "<" | | triangle_left | "H" | | hexagon2 | 4(CARETLEFT) | | caretleft |
| ">" | | triangle_right | "+" | | plus | 5(CARETRIGHT) | | caretright |
| "1" | | tri_down | "x" | | x | 6(CARETUP) | | caretup |
| "2" | | tri_up | "X" | | x (filled) | 7(CARETDOWN) | | caretdown |
| "3" | | tri_left | "D" | | diamond | 8(CARETLEFTBASE) | | caretleft |
| "4" | | tri_right | "d" | | thin_diamond | 9(CARETRIGHTBASE) | | caretright |
| "8" | | octagon | "\|" | | vline | 10(CARETUPBASE) | | caretup |

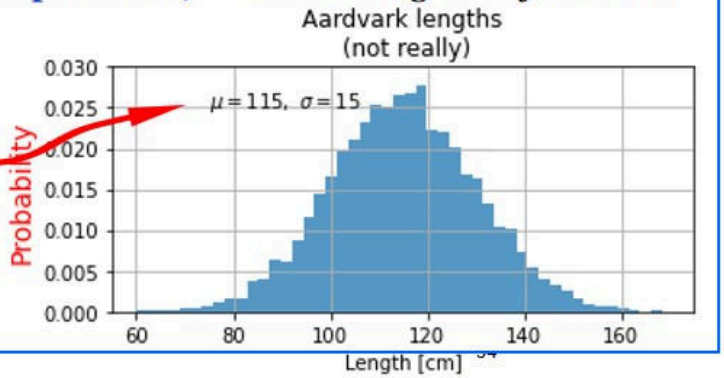

雨课堂 Rain Classroom

# A Brief matplotlib API Primer

## Labelling plots

- Axes labels and text
  - *set_xlabel*, *set_ylabel*, and *set_title* are used to add text in the indicated locations

```
mu, sigma = 115, 15
x = mu + sigma * np.random.randn(10000)
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
n, bins, patches = ax.hist(x, 50, density=1, facecolor='C0', alpha=0.75)  # the histogram of the data
ax.set_xlabel('Length [cm]')
ax.set_ylabel('Probability', fontsize=14, color='red')
ax.set_title('Aardvark lengths\n (not really)')
ax.text(75, .025, r'$\mu=115,\ \sigma=15$')
ax.axis([55, 175, 0, 0.03])
ax.grid(True);
```



# A Brief matplotlib API Primer

## Annotations

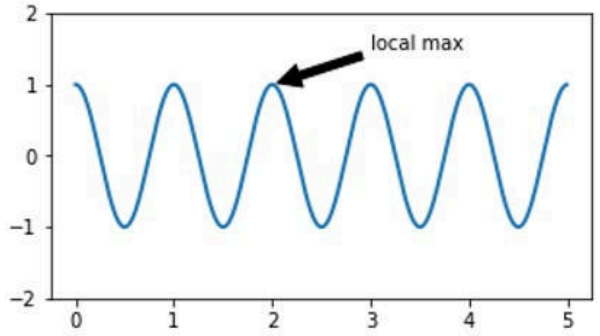
- We can annotate points on a plot, often by connecting an arrow pointing to *xy*, to a piece of text at *xytext*:

```
fig, ax = plt.subplots(figsize=(5, 2.7))

t = np.arange(0.0, 5.0, 0.01)
s = np.cos(2 * np.pi * t)
line, = ax.plot(t, s, lw=2)

ax.annotate('local max', xy=(2, 1), xytext=(3, 1.5),
            arrowprops=dict(facecolor='black', shrink=0.05))

ax.set_ylim(-2, 2);
```

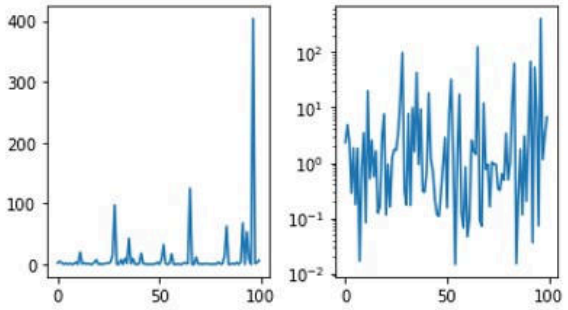


# A Brief matplotlib API Primer

## Axis scales and ticks

- Each Axes has two (or three) Axis objects representing the *x*- and *y-axis*. These control the scale of the Axis, the tick locators and the tick formatters.

```
data1, data2, data3, data4 = np.random.randn(4, 100)      # make 4 random data sets
fig, axs = plt.subplots(1, 2, figsize=(5, 2.7), layout='constrained')
xdata = np.arange(len(data1))   # make an ordinal for this
data = 10**data1
axs[0].plot(xdata, data)
axs[1].set_yscale('log')
axs[1].plot(xdata, data)
```




- The scale sets the mapping from data values to spacing along the Axis. This happens in both directions (maps from data coordinates坐标 to Axes, Figure, or screen coordinates).

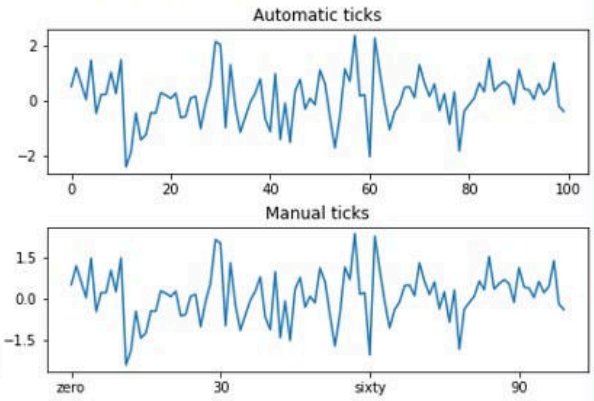
# A Brief matplotlib API Primer

## Tick locators and formatters

- Each Axis has a tick *locator* and *formatter* that choose where along the Axis objects to put tick marks. A simple interface to this is **set_xticks**:

```
data1, data2, data3, data4 = np.random.randn(4, 100)    # make 4 random data sets
xdata = np.arange(len(data1))  # make an ordinal for this
fig, axs = plt.subplots(2, 1, layout='constrained')
axs[0].plot(xdata, data1)
axs[0].set_title('Automatic ticks')

axs[1].plot(xdata, data1)
axs[1].set_xticks(np.arange(0, 100, 30), ['zero', '30', 'sixty', '90'])
axs[1].set_yticks([-1.5, 0, 1.5])  # note that we don't need to specify labels
axs[1].set_title('Manual ticks');
```



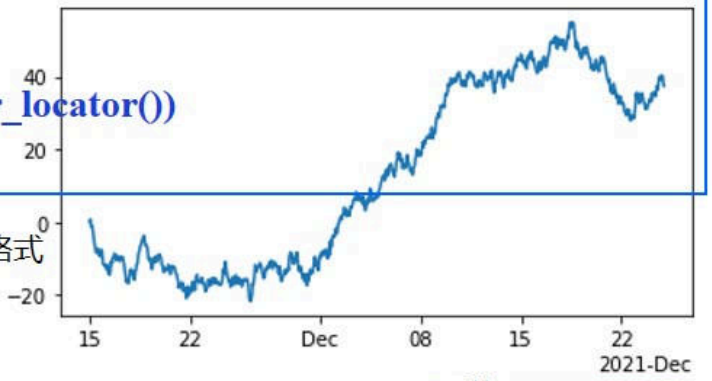

37

# A Brief matplotlib API Primer

## Plotting dates and strings

- Matplotlib can handle plotting arrays of *dates* and arrays of *strings*, as well as *floating* point numbers.(绘图数据不仅仅可以是数值型的，也可以是日期和字符串)
- For dates(日期):

```
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
dates = np.arange(np.datetime64('2021-11-15'), np.datetime64('2021-12-25'), np.timedelta64(1, 'h'))
data = np.cumsum(np.random.randn(len(dates)))
ax.plot(dates, data)
cdf = mpl.dates.ConciseDateFormatter(ax.xaxis.get_major_locator())
ax.xaxis.set_major_formatter(cdf)
```




- ConciseDateFormatter：找出能让日期更紧凑且完整的最佳格式
- set_major_formatter：设置轴的主格式





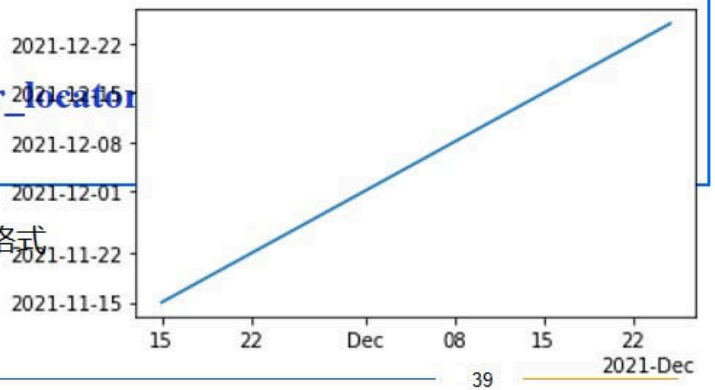
雨课堂 Rain Classroom

# A Brief matplotlib API Primer

## Plotting dates and strings

- Matplotlib can handle plotting arrays of *dates* and arrays of *strings*, as well as *floating* point numbers.(绘图数据不仅仅可以是数值型的，也可以是日期和字符串)
- For dates(日期):

```
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
dates = np.arange(np.datetime64('2021-11-15'), np.datetime64('2021-12-25'), np.timedelta64(1, 'h'))
data = np.cumsum(np.random.randn(len(dates)))
ax.plot(dates, dates)
cdf = mpl.dates.ConciseDateFormatter(ax.xaxis.get_major_locator
ax.xaxis.set_major_formatter(cdf)
```




- ConciseDateFormatter：找出能让日期更紧凑且完整的最佳格式
- set_major_formatter：设置轴的主格式





雨课堂 Rain Classroom

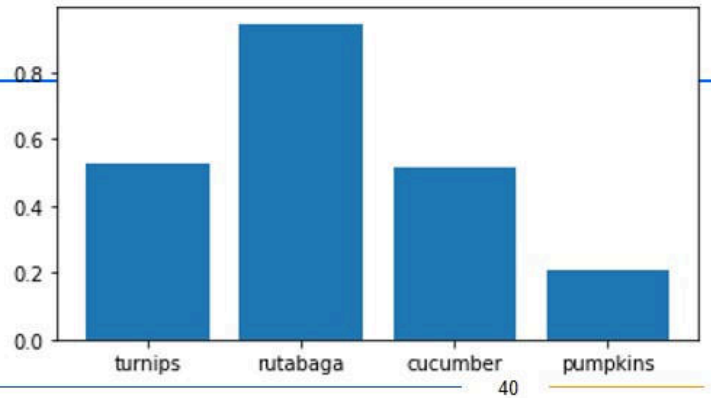
# A Brief matplotlib API Primer

## Plotting dates and strings

- Matplotlib can handle plotting arrays of *dates* and arrays of *strings*, as well as *floating* point numbers.
- For strings:

```
fig, ax = plt.subplots(figsize=(5, 2.7), layout='constrained')
categories = ['turnips', 'rutabaga', 'cucumber', 'pumpkins']

ax.bar(categories, np.random.rand(len(categories)))
```



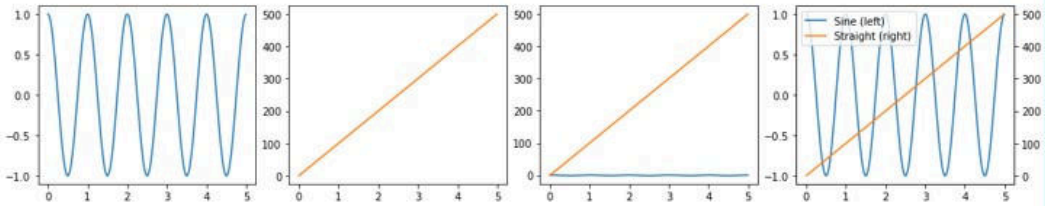

40

# A Brief matplotlib API Primer

## Additional Axis objects

- Plotting data of different magnitude in one chart may require an additional y-axis.

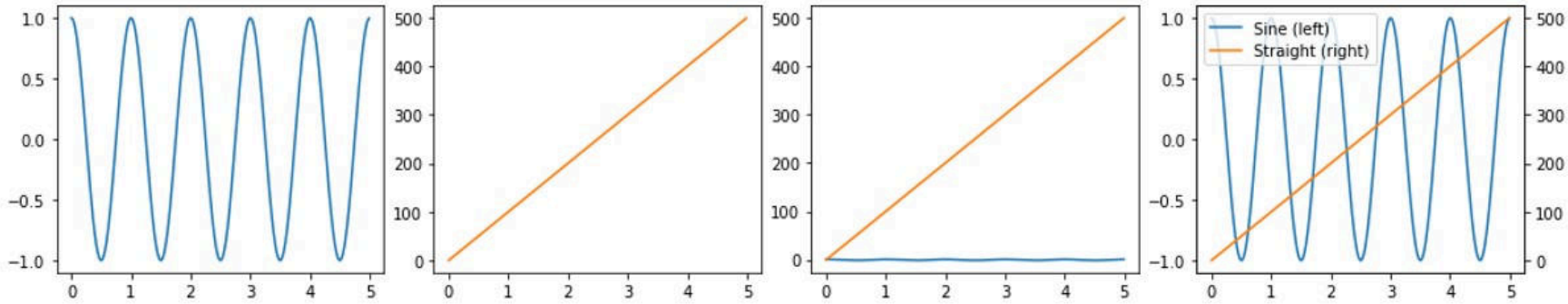
```
r = np.arange(0.0, 5.0, 0.01)      # 500  [0.0, 5.0)
s = np.cos(2*np.pi*r)              # 500  [0.0, 31.4)
t = range(len(r))                  # 500 [0,500)
fig, (ax1, ax2, ax3, ax4) = plt.subplots(1, 4, figsize=(14, 2.7), layout='constrained')
ax1.plot(r, s);        ax2.plot(r, t, 'C1')
ax3.plot(r, s);        ax3.plot(r, t, 'C1')
l1, = ax4.plot(r, s)
ax5 = ax4.twinx()
l2, = ax5.plot(r, t, 'C1')
ax5.legend([l1, l2], ['Sine (left)', 'Straight (right)'])
```



# A Brief matplotlib API Primer

## Additional Axis objects

- Plotting data of different magnitude in one chart may require an additional y-axis.



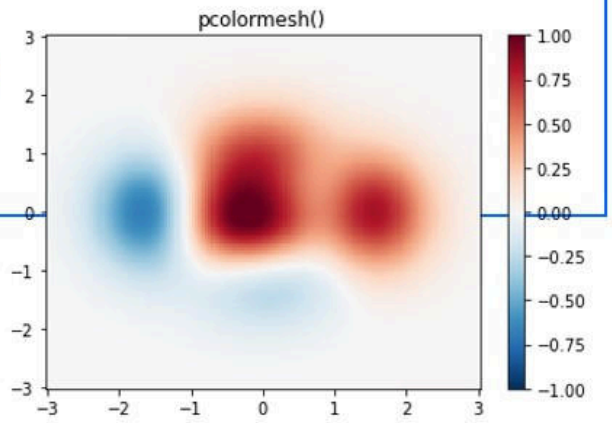
# A Brief matplotlib API Primer

## Color mapped data

- matplotlib can use color to represent the third dimension in the plot.

```
X, Y = np.meshgrid(np.linspace(-3, 3, 128), np.linspace(-3, 3, 128))
Z = (1 - X/2 + X**5 + Y**3) * np.exp(-X**2 - Y**2)

fig, ax = plt.subplots()
pc = ax.pcolormesh(X, Y, Z, vmin=-1, vmax=1, cmap='RdBu_r')
fig.colorbar(pc, ax=ax)
ax.set_title('pcolormesh()')
```



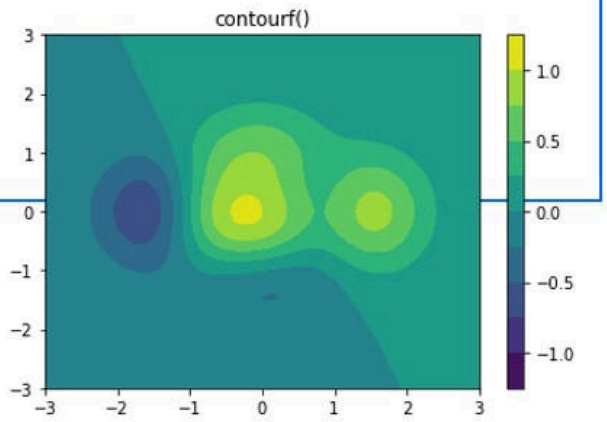
# A Brief matplotlib API Primer

## Color mapped data

- matplotlib can use color to represent the third dimension in the plot.

```
X, Y = np.meshgrid(np.linspace(-3, 3, 128), np.linspace(-3, 3, 128))
Z = (1 - X/2 + X**5 + Y**3) * np.exp(-X**2 - Y**2)

fig, ax = plt.subplots()
co = ax.contourf(X, Y, Z, levels=np.linspace(-1.25, 1.25, 11))
fig.colorbar(co, ax=ax)
ax.set_title('contourf()')
```



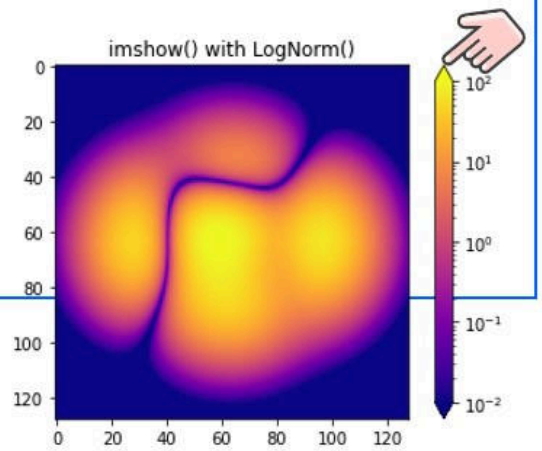
# A Brief matplotlib API Primer

## Color mapped data

- matplotlib can use color to represent the third dimension in the plot.

```
X, Y = np.meshgrid(np.linspace(-3, 3, 128), np.linspace(-3, 3, 128))
Z = (1 - X/2 + X**5 + Y**3) * np.exp(-X**2 - Y**2)

fig, ax = plt.subplots()
pc = ax.imshow(Z**2 * 100, cmap='plasma',
               norm=mpl.colors.LogNorm(vmin=0.01, vmax=100))
fig.colorbar(pc, ax=ax, extend='both')
ax.set_title('imshow() with LogNorm()')
```



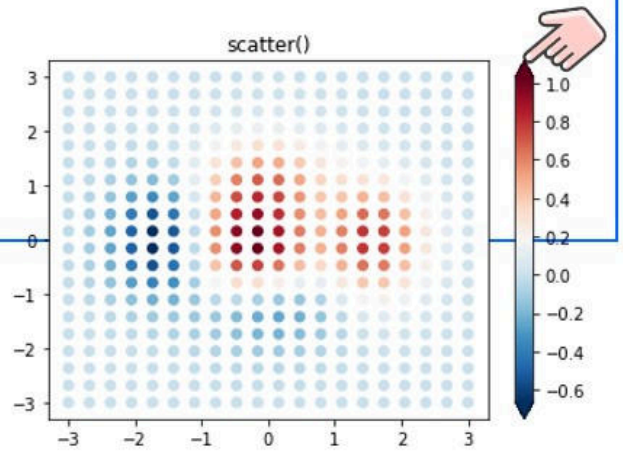
# A Brief matplotlib API Primer

## Color mapped data

- matplotlib can use color to represent the third dimension in the plot.

```
X, Y = np.meshgrid(np.linspace(-3, 3, 20), np.linspace(-3, 3, 20))        # 取少点数值
Z = (1 - X/2 + X**5 + Y**3) * np.exp(-X**2 - Y**2)

fig, ax = plt.subplots()
pc = ax.scatter(X, Y, c=Z, cmap='RdBu_r')
fig.colorbar(pc, ax=ax, extend='both')
ax.set_title('scatter()')
```



# Customizing Matplotlib with rcParams

## Runtime rc settings

- We can dynamically change the default rc (runtime configuration) settings in a python script or interactively from the python shell.
- All rc settings are stored in a dictionary-like variable called *matplotlib.rcParams*, which is **global** to the matplotlib package.
- There are three ways to customize Matplotlib:
  - Setting rcParams at runtime. 运行时动态修改
  - Using style sheets. 使用样式表（已有模板或自定义模板）
  - Changing your matplotlibrc file. 直接修改配置文件

47


雨课堂 Rain Classroom

# The list of rcParams

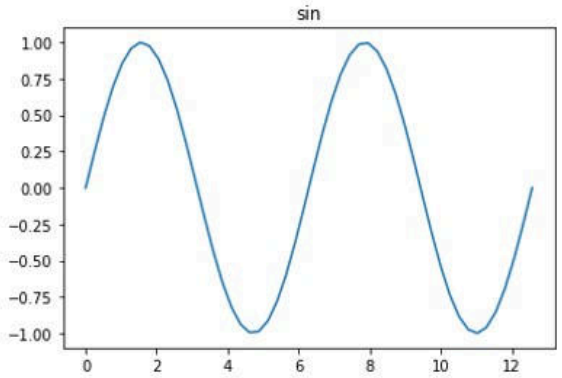
_internal.classic_mode
agg.path.chunksize
animation.bitrate
animation.codec
animation.convert_args
animation.convert_path
animation.embed_limit
animation.ffmpeg_args
animation.ffmpeg_path
animation.frame_format
animation.html
animation.writer
axes.autolimit_mode
axes.axisbelow
axes.edgecolor
axes.facecolor
axes.formatter.limits
axes.formatter.min_expon
axes.formatter.offset_thre
axes.formatter.use_locale
axes.formatter.use_matht
axes.formatter.useoffset
axes.grid
axes.grid.axis
axes.grid.which
axes.labelcolor
axes.labelpad
axes.labelsize
axes.labelweight
axes.linewidth
axes.prop_cycle
axes.spines.bottom
axes.spines.left
axes.spines.right
axes.spines.top
axes.titlecolor
axes.titlelocation
axes.titlepad
axes.titlesize
axes.titleweight
axes.titley
axes.unicode_minus
axes.xmargin
axes.ymargin
axes.zmargin
axes3d.grid
backend
backend_fallback
boxplot.bootstrap
boxplot.boxprops.color
boxplot.boxprops.linestyl
boxplot.boxprops.linewid
boxplot.capprops.color
boxplot.capprops.linestyl
boxplot.capprops.linewid
boxplot.flierprops.color
boxplot.flierprops.linestyl
boxplot.flierprops.linewid
boxplot.flierprops.marker
boxplot.flierprops.marker
boxplot.flierprops.marker
boxplot.flierprops.marker
boxplot.meanline
boxplot.meanprops.color
boxplot.meanprops.linest
boxplot.meanprops.linew
boxplot.meanprops.mark
boxplot.meanprops.mark
boxplot.meanprops.mark
boxplot.meanprops.mark
boxplot.medianprops.col
boxplot.medianprops.line
boxplot.medianprops.line
boxplot.notch
boxplot.patchartist
boxplot.showbox
boxplot.showcaps
boxplot.showfliers
boxplot.showmeans
boxplot.vertical
boxplot.whiskerprops.col
boxplot.whiskerprops.line
boxplot.whiskerprops.line
boxplot.whiskers
contour.corner_mask
contour.linewidth
contour.negative_linestyl
date.autoformatter.day
date.autoformatter.hour
date.autoformatter.micro
date.autoformatter.minut
date.autoformatter.mont
date.autoformatter.secon
date.autoformatter.year
date.converter
date.epoch
date.interval_multiples
docstring.hardcopy
errorbar.capsize
figure.autolayout
figure.constrained_layout.
figure.constrained_layout.
figure.constrained_layout.
figure.constrained_layout.
figure.constrained_layout.
figure.dpi
figure.edgecolor
figure.facecolor
figure.figsize
figure.frameon
figure.max_open_warning
figure.raise_window
figure.subplot.bottom
figure.subplot.hspace
figure.subplot.left
figure.subplot.right
figure.subplot.top
figure.subplot.wspace
figure.titlesize
figure.titleweight
font.cursive
font.family
font.fantasy
font.monospace
font.sans-serif
font.serif
font.size
font.stretch
font.style
font.variant
font.weight
grid.alpha
grid.color
grid.linestyle
grid.linewidth
hatch.color
hatch.linewidth
hist.bins
image.aspect
image.cmap
image.composite_image
image.interpolation
image.lut
image.origin
image.resample
interactive
keymap.back
keymap.copy
keymap.forward
keymap.fullscreen
keymap.grid
keymap.grid_minor
keymap.help
keymap.home
keymap.pan
keymap.quit
keymap.quit_all
keymap.save
keymap.xscale
keymap.yscale
keymap.zoom
legend.borderaxespad
legend.borderpad
legend.columnspacing
legend.edgecolor
legend.facecolor
legend.fancybox
legend.fontsize
legend.framealpha
legend.frameon
legend.handleheight
legend.handlelength
legend.handletextpad
legend.labelcolor
legend.labelspacing
legend.loc
legend.markerscale
legend.numpoints
legend.scatterpoints
legend.shadow
legend.title_fontsize
lines.antialiased
lines.color
lines.dash_capstyle
lines.dash_joinstyle
lines.dashdot_pattern
lines.dashed_pattern
lines.dotted_pattern
lines.linestyle
lines.linewidth
lines.marker
lines.markeredgecolor
lines.markeredgewidth
lines.markerfacecolor
lines.markersize
lines.scale_dashes
lines.solid_capstyle
lines.solid_joinstyle
markers.fillstyle
mathtext.bf
mathtext.cal
mathtext.default
mathtext.fallback
mathtext.fontset
mathtext.it
mathtext.rm
mathtext.sf
mathtext.tt
patch.antialiased
patch.edgecolor
patch.facecolor
patch.force_edgecolor
patch.linewidth
path.effects
path.simplify
path.simplify_threshold
path.sketch
path.snap
pcolor.shading
pcolormesh.snap
pdf.compression
pdf.fonttype
pdf.inheritcolor
pdf.use14corefonts
pgf.preamble
pgf.rcfonts
pgf.texsystem
polaraxes.grid
ps.distiller.res
ps.fonttype
ps.papersize
ps.useafm
ps.usedistiller
savefig.bbox
savefig.directory
savefig.dpi
savefig.edgecolor
savefig.facecolor
savefig.format
savefig.orientation
savefig.pad_inches
savefig.transparent
scatter.edgecolors
scatter.marker
svg.fonttype
svg.hashsalt
svg.image_inline
text.antialiased
text.color
text.hinting
text.hinting_factor
text.kerning_factor
text.latex.preamble
text.usetex
timezone
tk.window_focus
toolbar
webagg.address
webagg.open_in_browser
webagg.port
webagg.port_retries
xaxis.labellocation
xtick.alignment
xtick.bottom
xtick.color
xtick.direction
xtick.labelbottom
xtick.labelcolor
xtick.labelsize
xtick.labeltop
xtick.major.bottom
xtick.major.pad
xtick.major.size
xtick.major.top
xtick.major.width
xtick.minor.bottom
xtick.minor.pad
xtick.minor.size
xtick.minor.top
xtick.minor.visible
xtick.minor.width
xtick.right
yaxis.labellocation
ytick.alignment
ytick.color
ytick.direction
ytick.labelcolor
ytick.labelleft
ytick.labelright
ytick.labelsize
ytick.left
ytick.major.left
ytick.major.pad
ytick.major.right
ytick.major.size
ytick.major.width
ytick.minor.left
ytick.minor.pad
ytick.minor.right
ytick.minor.size
ytick.minor.visible
ytick.minor.width
ytick.right

48

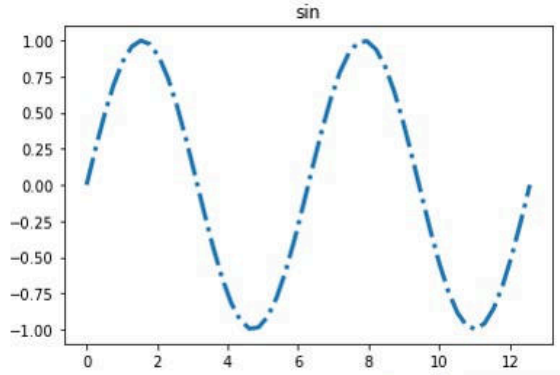
# Customizing Matplotlib with rcParams

## Example

```
x = np.linspace(0, 4*np.pi)
y = np.sin(x)
plt.plot(x, y, label="$sin(x)$")
plt.title('sin')
plt.show()
```




```
# Change setting
plt.rcParams['lines.linestyle'] = '-.'
plt.rcParams['lines.linewidth'] = 3
plt.plot(x, y, label="$sin(x)$")
plt.title('sin')
plt.show()
```



# Customizing Matplotlib with rcParams

## rcParams for lines

| rcParams | Description | scope |
|---|---|---|
| lines.linewidth | line width | [0-10], default: 1.5 |
| lines.linestyle | line style | "-" "--" "-." ":" , default: "-" |
| lines.marker | shape of point on line | "o" "D" "h" "." "," "S" , etc. default: None。 |
| lines.markersize | size of point | [0-10], default: 1 |

# Customizing Matplotlib with rcParams

| marker value | Description | marker value | Description |
|---|---|---|---|
| 'o' | 圆圈 | '.' | 点 |
| 'D' | 菱形 | 's' | 正方形 |
| 'h' | 六边形1 | '*' | 星号 |
| 'H' | 六边形2 | 'd' | 小菱形 |
| '-' | 水平线 | 'v' | 一角朝下的三角形 |
| '8' | 八边形 | '<' | 一角朝左的三角形 |
| 'p' | 五边形 | '>' | 一角朝右的三角形 |
| ',' | 像素 | '^' | 一角朝上的三角形 |
| '+' | 加号 | '\' | 竖线 |
| 'None' | 无 | 'x' | X |

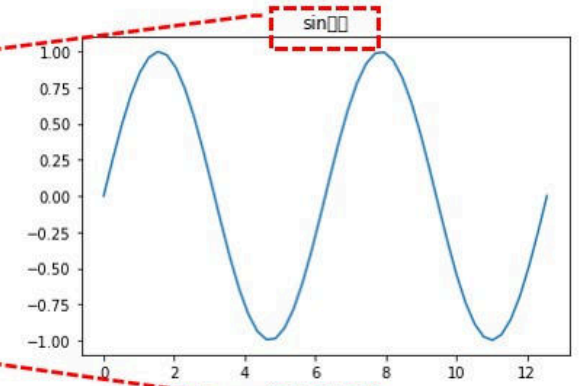
# Customizing Matplotlib with rcParams

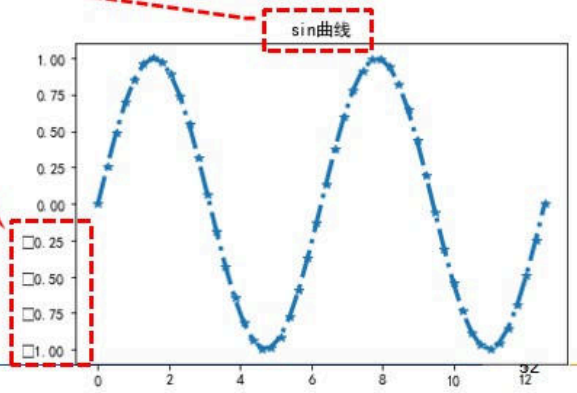
**Example**(显示中文字体)

```
plt.plot(x,y,label="$sin(x)$")
plt.title('sin曲线')
plt.show()

plt.rcParams['font.sans-serif'] = 'SimHei'
plt.rcParams['lines.marker'] = '*'
plt.plot(x,y,label="$sin(x)$")
plt.title('sin曲线')
plt.show()

plt.rcParams['axes.unicode_minus'] = False
```



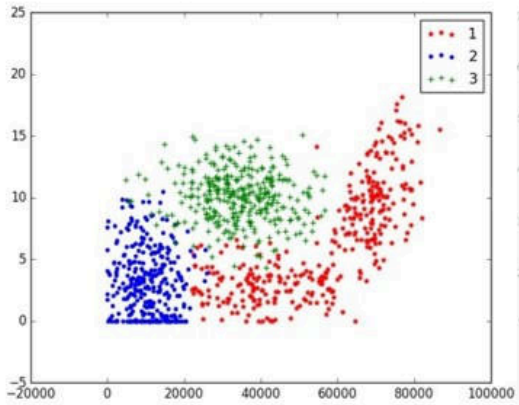



# 目录



# Plot types

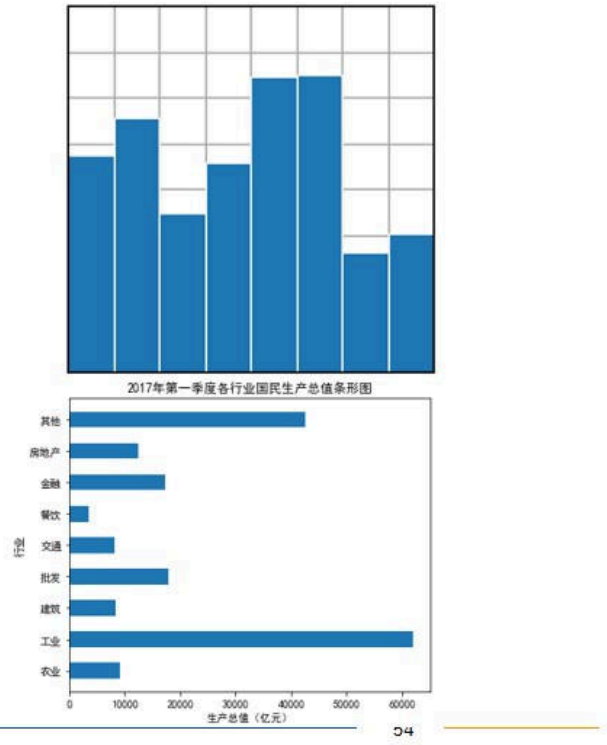
## scatter(x,y)




## plot(x,y)

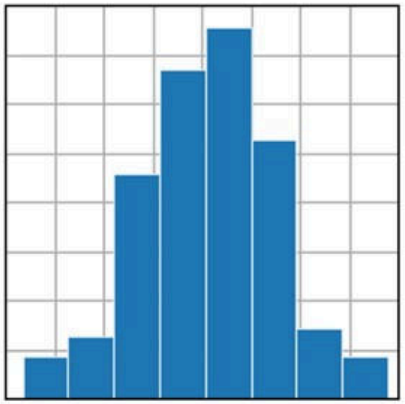


## bar(x,height)/barh(y,width)

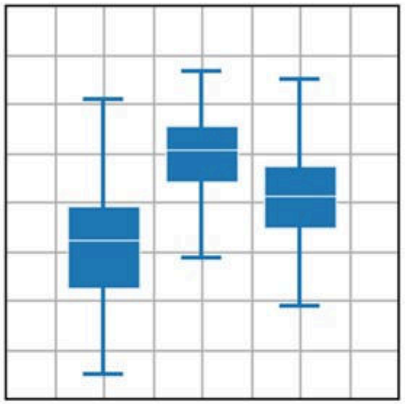


# Plot types

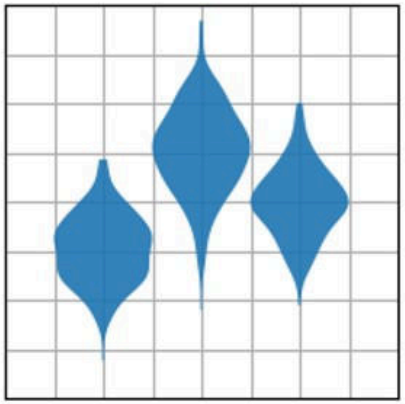
hist(x)



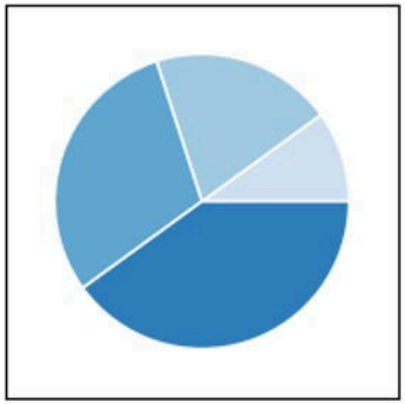
boxplot(X)



violinplot(D)



pie(x)

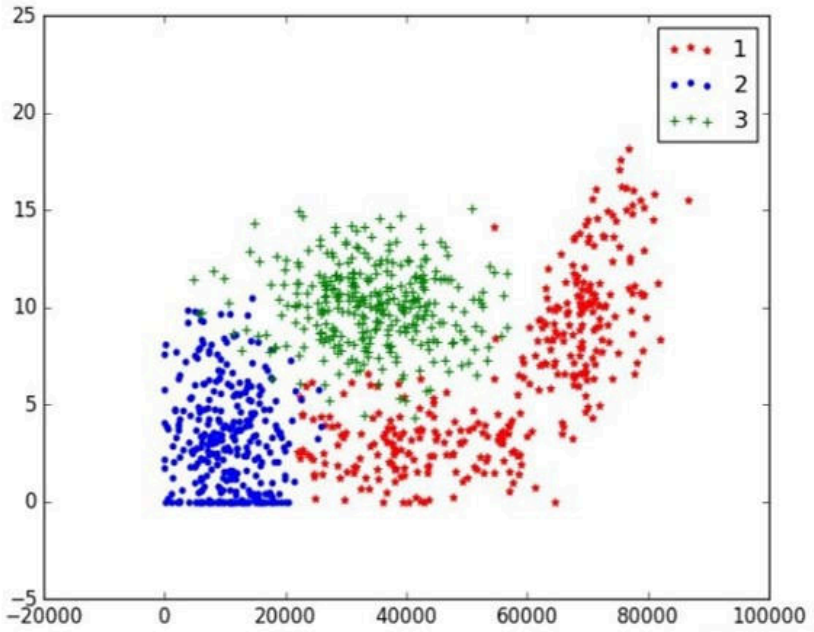


55

# Basic plot types

## scatter

- A scatter plot of y vs. x with varying marker size and/or color.
- 通常用于比较跨类别的数据
  - 值：由点在图表中的位置表示
  - 类别：由图表中的不同标记表示



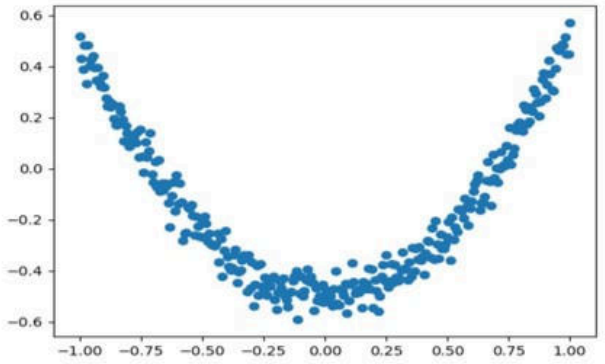
# Basic plot types

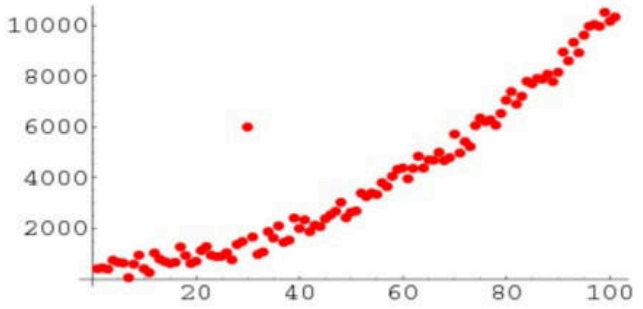
## scatter

- 散点图可以提供两类关键信息。

(1)特征之间是否存在数值或者数量的关联趋势，关联趋势是线性的还是非线性的。

(2)如果某一个点或者某几个点偏离大多数点，则这些点就是离群值、通过散点图可以一自了然，从而可以进一步分析这些离群值是否在建模分析中产生很大的影响。



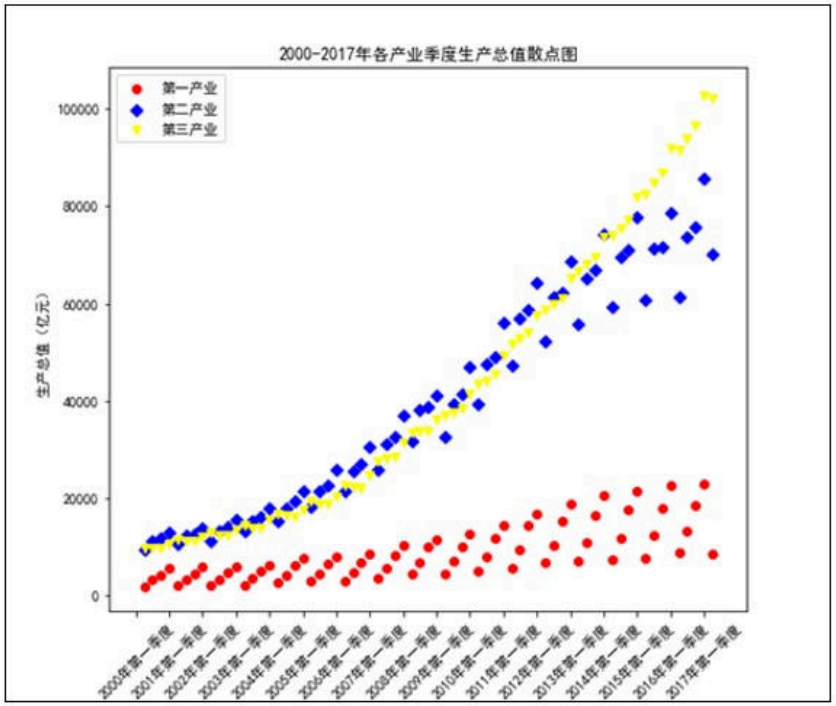




57

# Basic plot types

## scatter

- 散点图通过散点的疏密程度和变化趋势表示两个特征的数量关系。
- 如果有3个特征、其中一个特征为类别型，散点图改变不同特征的点的形状或者颜色，即可了解两个数值型特征和这个类别型之间的关系。








雨课堂 Rain Classroom

# Basic plot types

## scatter

*matplotlib.pyplot.scatter(x, y, s=None, c=None, marker=None, alpha=None, **kwargs)*

| parameter | Description |
|---|---|
| x, y | The data positions. *float or array-like, shape (n, )* |
| s | The marker size in points**2. Default is rcParams['lines.markersize'] ** 2.<br>*float or array-like, shape (n, ), optional* |
| c | The marker colors. Possible values: A scalar or sequence of n numbers; A 2D array; A sequence of colors; A single color format string. |
| marker | The marker style. marker can be either an instance of the class or the text shorthand for a particular marker. |
| alpha | The alpha blending value, between 0 (transparent透明) and 1 (opaque不透明) |


雨课堂 Rain Classroom

# scatter

**Example：** 使用scatter函数绘制2000~2017年各季度国民生产总值散点图

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = 'SimHei'          # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False          # 设置负号可显示
data = np.load('国民经济核算季度数据.npz', allow_pickle=True)
name = data['columns']                              # 提取其中的columns数组，视为数据的标签
values = data['values']                             # 提取其中的values数组，数据的存在位置
```

# scatter

```
>>> name
```

```
array(['序号', '时间', '国内生产总值_当季值(亿元)', '第一产业增加值_当季值(亿元)', '第二产业增加值_当季值(亿元)',
'第三产业增加值_当季值(亿元)', '农林牧渔业增加值_当季值(亿元)', '工业增加值_当季值(亿元)', '建筑业增加值_当
季值(亿元)', '批发和零售业增加值_当季值(亿元)', '交通运输、仓储和邮政业增加值_当季值(亿元)', '住宿和餐饮业
增加值_当季值(亿元)', '金融业增加值_当季值(亿元)', '房地产业增加值_当季值(亿元)', '其他行业增加值_当季值(亿
元)'], dtype=object)
```

```
>>> values
```

```
array([[1,   '2000年第一季度', 21329.9, ..., 1235.9, 933.7, 3586.1],
       [2,   '2000年第二季度', 24043.4, ..., 1124.0, 904.7, 3464.9],
       [3,   '2000年第三季度', 25712.5, ..., 1170.4, 1070.9, 3518.2],
        ...,
       [67, '2016年第三季度', 190529.5, ..., 15472.5, 12164.1, 37964.1],
       [68, '2016年第四季度', 211281.3, ..., 15548.7, 13214.9, 39848.4],
       [69, '2017年第一季度', 180682.7, ..., 17213.5, 12393.4, 42443.1]],
       dtype=object)
```

# scatter

**Example：** 使用scatter函数绘制2000~2017年各季度国民生产总值散点图

```
plt.figure(figsize=(8,7))                                          # 设置画布
plt.scatter(values[:,0], values[:, 2], marker='o')                 # 绘制散点图
plt.xlabel('年份')                                                  # 添加横轴标签
plt.ylabel('生产总值（亿元）')                                       # 添加y轴名称
plt.xticks(range(0,70,4), values[range(0,70,4),1], rotation=45)    #指定x轴刻度与标签
plt.title('2000-2017年季度生产总值散点图')                           # 添加图表标题
plt.savefig('2000-2017年季度生产总值散点图.png')
plt.show()
```

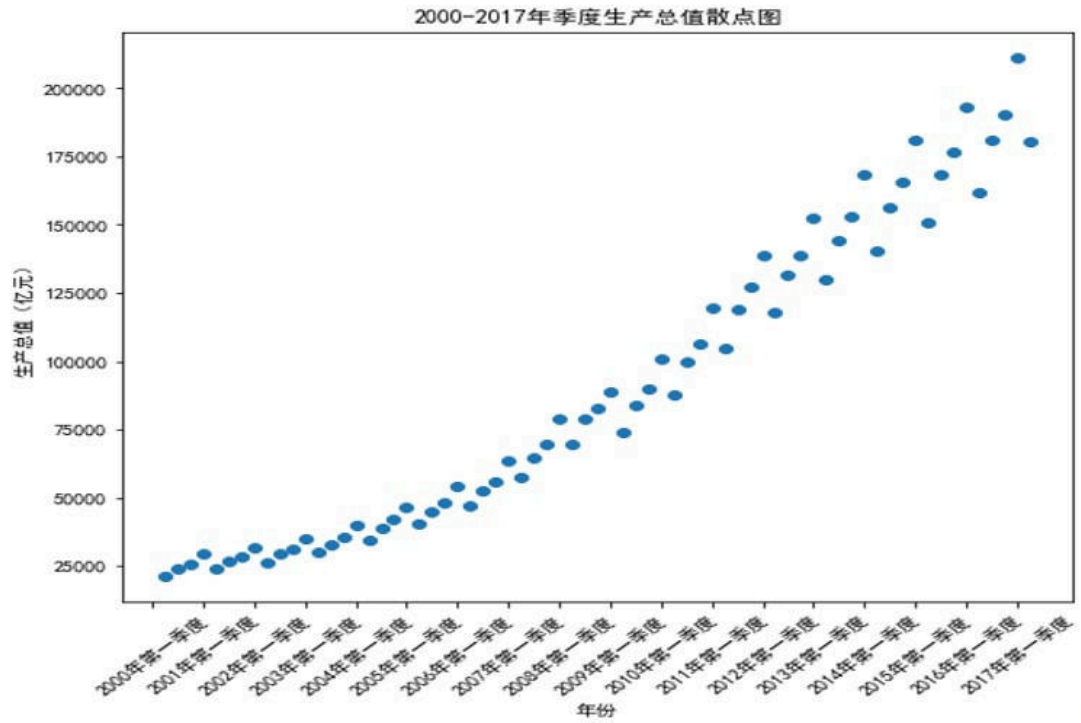




雨课堂 Rain Classroom

# scatter

**Example：** 使用scatter函数绘制2000~2017年各季度国民生产总值散点图








雨课堂 Rain Classroom

# scatter

**Example：** 使用scatter函数绘制2000~2017年间各产业各季度国民生产总值散点图

```
plt.figure(figsize=(8,7))                                          # 设置画布
plt.scatter(values[:,0],values[:,3], marker='o',c='red')           # 绘制散点1
plt.scatter(values[:,0],values[:,4], marker='D',c='blue')          # 绘制散点2
plt.scatter(values[:,0],values[:,5], marker='v',c='green')         # 绘制散点3
plt.xlabel('年份')                                                  # 添加横轴标签
plt.ylabel('生产总值（亿元）')                                       # 添加纵轴标签
plt.xticks(range(0,70,4),values[range(0,70,4),1],rotation=45)      # 指定x轴刻度与标签
plt.title('2000-2017年各产业季度生产总值散点图')                      # 添加图表标题
plt.legend(['第一产业','第二产业','第三产业'])                       # 添加图例
plt.savefig('tmp/2000-2017年各产业季度生产总值散点图.png')
plt.show()
```



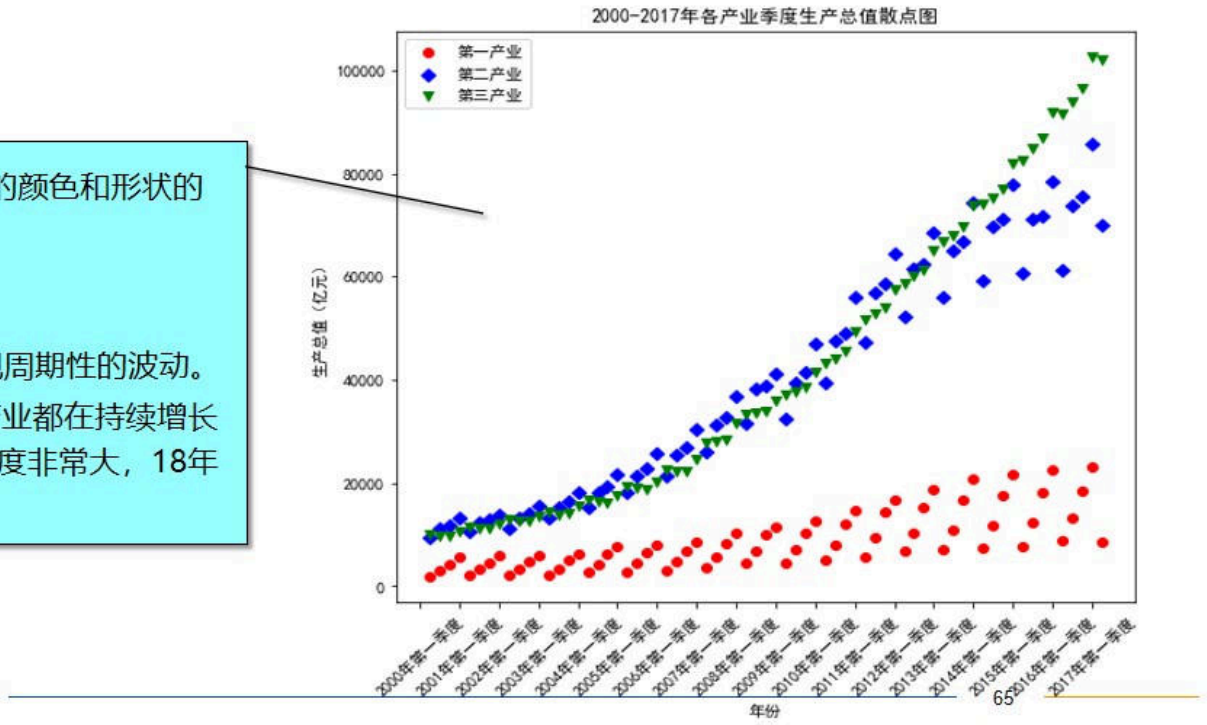


雨课堂 Rain Classroom

# scatter

**Example：** 使用scatter函数绘制2000~2017年间各产业各季度国民生产总值散点图

- 通过散点图进行数据分析



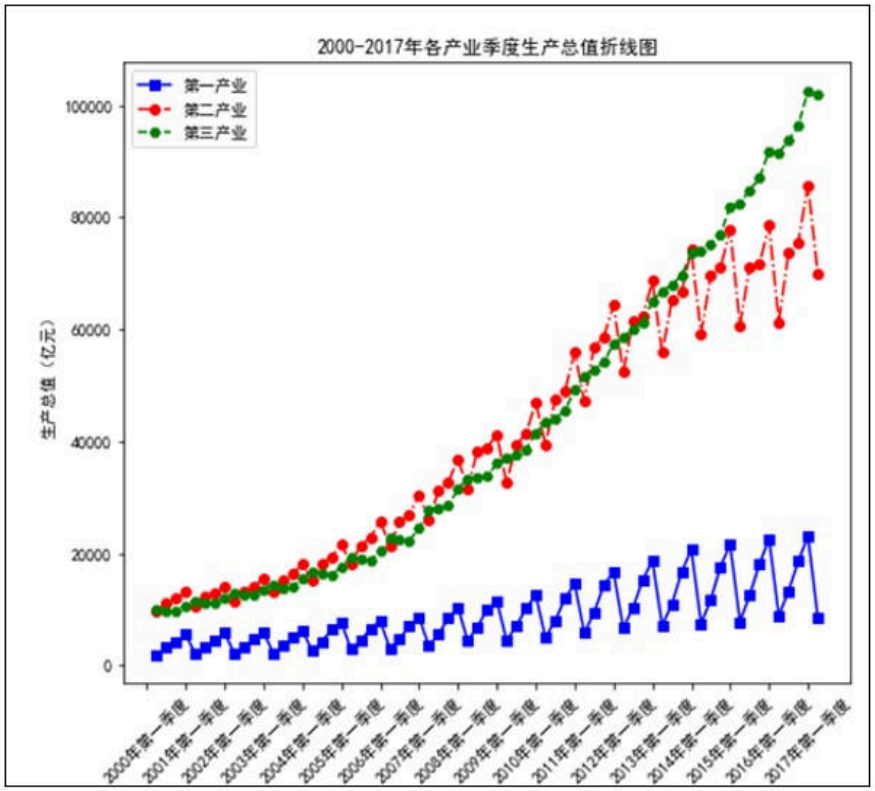

- 通过这段代码绘制的散点图中点的颜色和形状的区别可以看出：
  - 第一产业增长平缓，
  - 第三产业呈现指数型增长，
  - 第二产业每年会根据季度呈现周期性的波动。
- 总体看来，我国近18年的各个产业都在持续增长中，并且第二和第三产业增长幅度非常大，18年间增长了400%以上。

# Basic plot types

## plot

- Plot y versus x as lines and/or markers.
- The coordinates of the points or line nodes are given by x, y.

- 折线图的主要功能是查看因变量y随着自变量x改变的趋势，最适合用于显示随时间（根据常用比例设置）而变化的连续数据。同时还可以看出数量的差异，增长趋势的变化。



# Basic plot types

**plot**

***matplotlib.pyplot.plot(*args, scalex=True, scaley=True, data=None, **kwargs)***

| parameter | Description |
|---|---|
| x, y | array-like or scalar, x values are optional and default to range(len(y)) |
| scalex, scaley | ***bool, default: True.*** These parameters determine if the view limits are adapted to the data limits. (视图limits是否适应数据limits) |
| data | indexable object, optional. An object with labelled data. If given, provide the label names to plot in x and y. |
| **kwargs | to specify properties(optional) like a line label (for auto legends), linewidth, color, linestyle, marker, etc. |

# Basic plot types

## plot

- 其中，color参数的8种常用颜色的缩写。

| 颜色缩写 | 代表的颜色 | 颜色缩写 | 代表的颜色 |
|---|---|---|---|
| b | 蓝色 | m | 品红 |
| g | 绿色 | y | 黄色 |
| r | 红色 | k | 黑色 |
| c | 青色 | w | 白色 |





雨课堂 Rain Classroom

# plot

**Example：** 绘制2000-2017年各季度国民生产总值折线图

```
plt.figure(figsize=(8,7))
plt.plot(values[:,0],values[:,2],color = 'r',linestyle = '--')      # 绘制折线图
plt.xlabel('年份')                                                  # 添加横轴标签
plt.ylabel('生产总值（亿元）')                                      # 添加y轴名称
plt.xticks(range(0,70,4),values[range(0,70,4),1],rotation=45)       #指定x轴刻度与标签
plt.title('2000-2017年季度生产总值折线图')                          # 添加图表标题
plt.savefig('tmp/2000-2017年季度生产总值折线图.png')
plt.show()
```

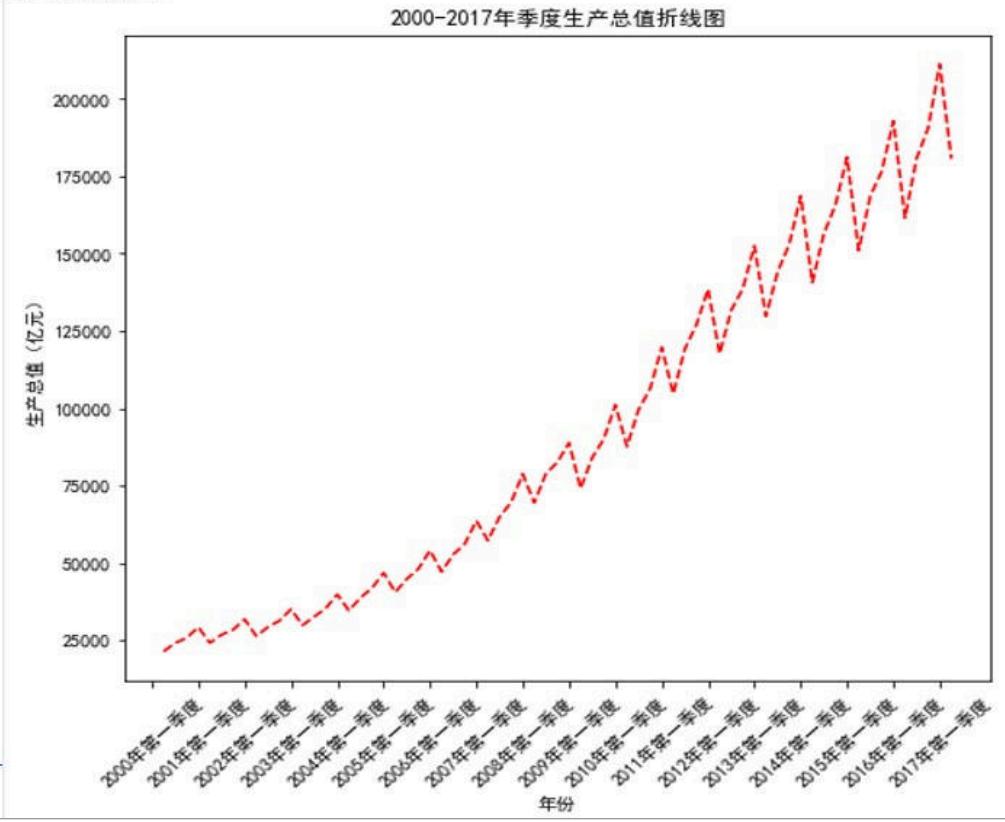




雨课堂 Rain Classroom

# plot

**Example：** 绘制2000-2017年各季度国民生产总值折线图



# plot

**Example：** 绘制2000-2017年各季度国民生产总值点线图

■ 使用marker参数可以绘制点线图，能够使图形更加丰富。

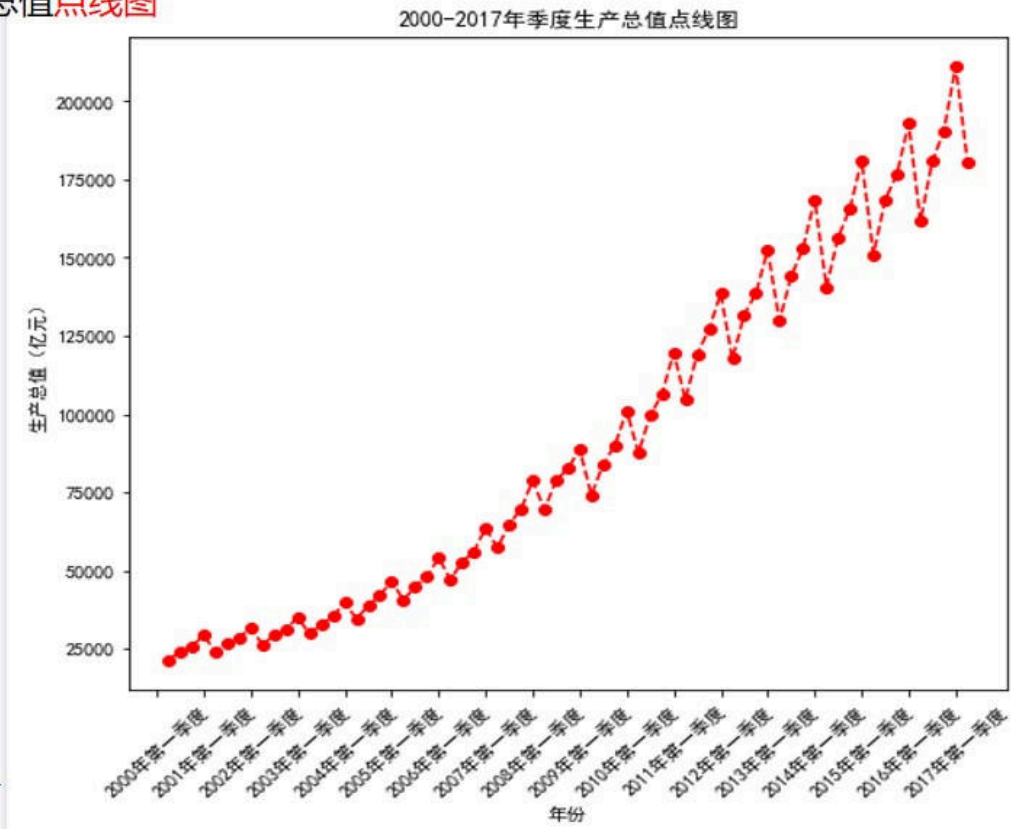
```
plt.figure(figsize=(8,7))
plt.plot(values[:,0],values[:,2],color = 'r',linestyle = '--',marker = 'o')   # 绘制折线图
plt.xlabel('年份')                                                            # 添加横轴标签
plt.ylabel('生产总值（亿元）')                                                 # 添加y轴名称
plt.xticks(range(0,70,4),values[range(0,70,4),1],rotation=45)
plt.title('2000-2017年季度生产总值点线图')                                     # 添加图表标题
plt.savefig('tmp/2000-2017年季度生产总值点线图.png')
plt.show()
```



雨课堂 Rain Classroom

# plot

**Example：** 绘制2000-2017年各季度国民生产总值点线图



# plot

**Example：** 绘制2000-2017年各产业各季度国民生产总值点线图

```
plt.figure(figsize=(8,7))
plt.plot(values[:, 0], values[:,3], 'bs-',
         values[:, 0], values[:,4], 'ro-.',
         values[:, 0], values[:,5], 'gH--')      # 绘制折线图
plt.xlabel('年份')                                # 添加横轴标签
plt.ylabel('生产总值（亿元）')                     # 添加y轴名称
plt.xticks(range(0,70,4),values[range(0,70,4),1],rotation=45)
plt.title('2000-2017年各产业季度生产总值折线图')# 添加图表标题
plt.legend(['第一产业','第二产业','第三产业'])    #添加图例
plt.savefig('tmp/2000-2017年季度各产业生产总值折线图.png')
plt.show()
```

- plot函数一次可以接收多组数据，添加多条折线图
- 同时可以分别定义每条折线的颜色、点的形状和类型
- 还可以将这3个参数连接在一起，用一个字符串表示。



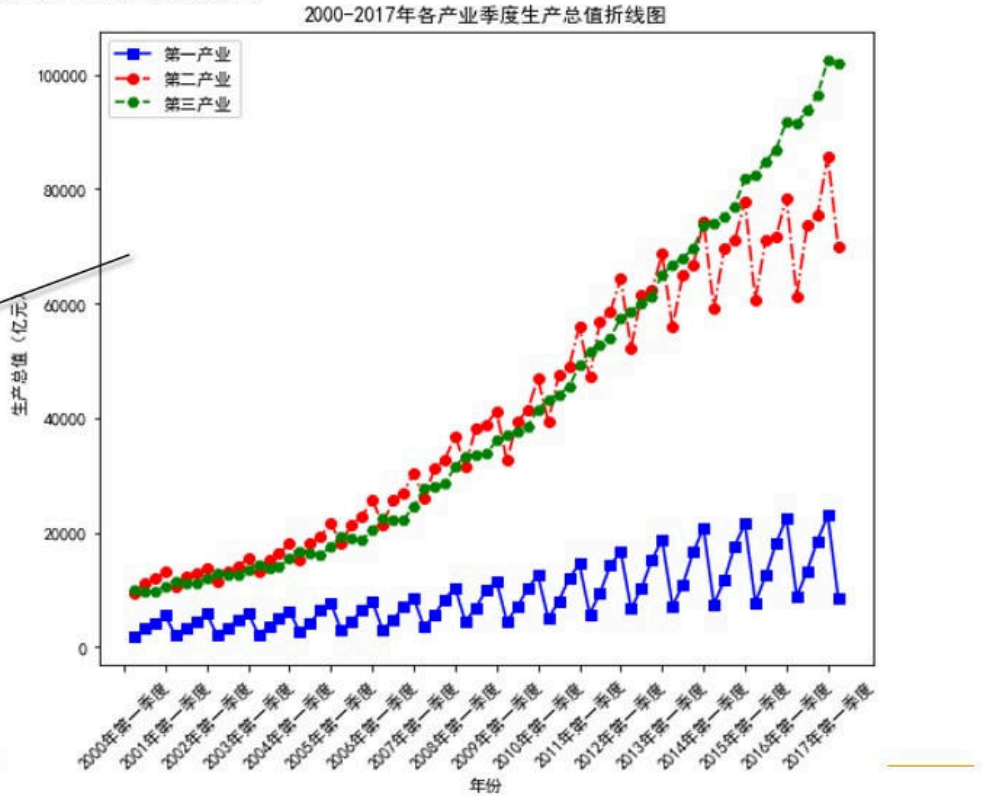


雨课堂 Rain Classroom

# plot

**Example：** 绘制2000-2017年各产业各季度国民生产总值点线图

- 通过折线图进行数据分析




■ 通过这张折线图看出，我国现阶段国民经济增长的主要动力是第三产业，其次是第二产业。

# 任务实现

## 1. 绘制2000-2017各产业与行业的国民生产总值散点图

- 国民生产总值数据总共有三大产业的国民生产总值，以及农业、工业、建筑、批发、交通、餐饮、金融、房地产和其他行业各个季度的增加值。

- 通过散点图分析三大行业的国民生产总值可以发现我国产业结构。
- 通过比较各行业间季度的增加值则可以发现国民经济的主要贡献行业。

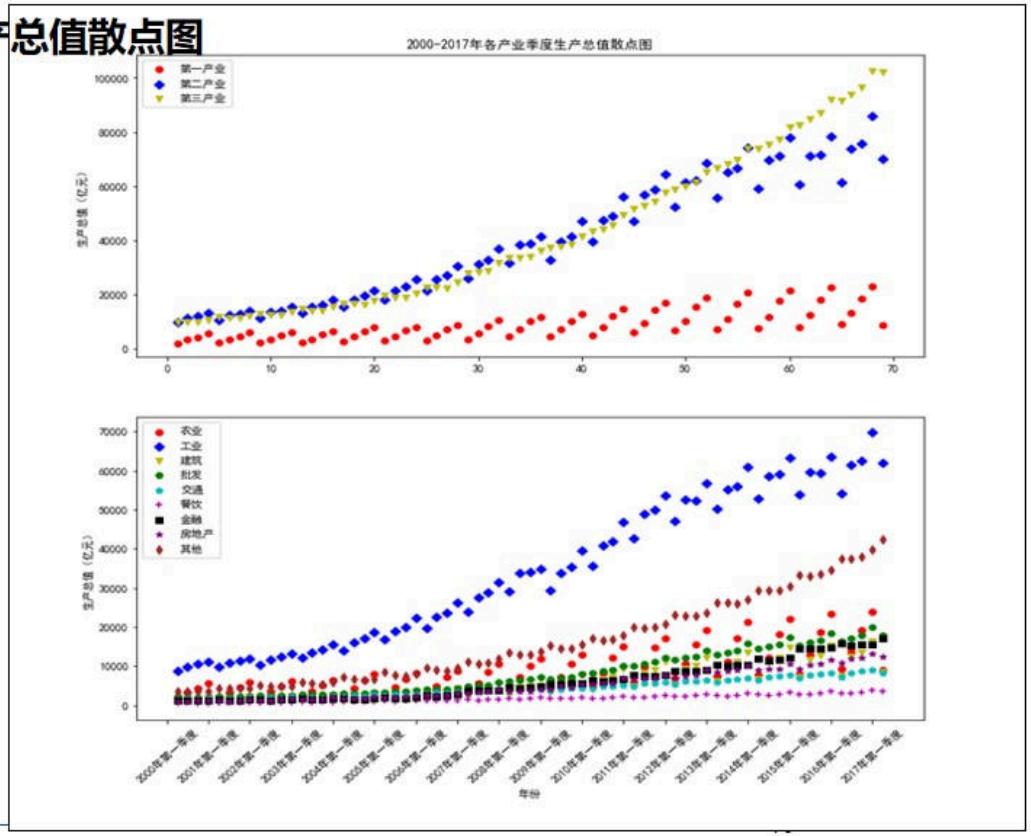




雨课堂 Rain Classroom

# 任务实现

## 1. 绘制2000-2017各产业与行业的国民生产总值散点图



# 任务实现

## 1. 绘制2000-2017各产业与行业的国民生产总值散点图

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = 'SimHei'                    # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False                    # 设置负号显示
data = np.load('国民经济核算季度数据.npz', allow_pickle=True)    # 读取数据文件
name = data['columns']                                        # 提取其中的columns数组，视为数据的索引
values = data['values']                                       # 提取其中的values数组，即数据
p = plt.figure(figsize=(12,12))                               # 创建画布
```





雨课堂 Rain Classroom

# 任务实现

## 1. 绘制2000-2017各产业与行业的国民生产总值散点图

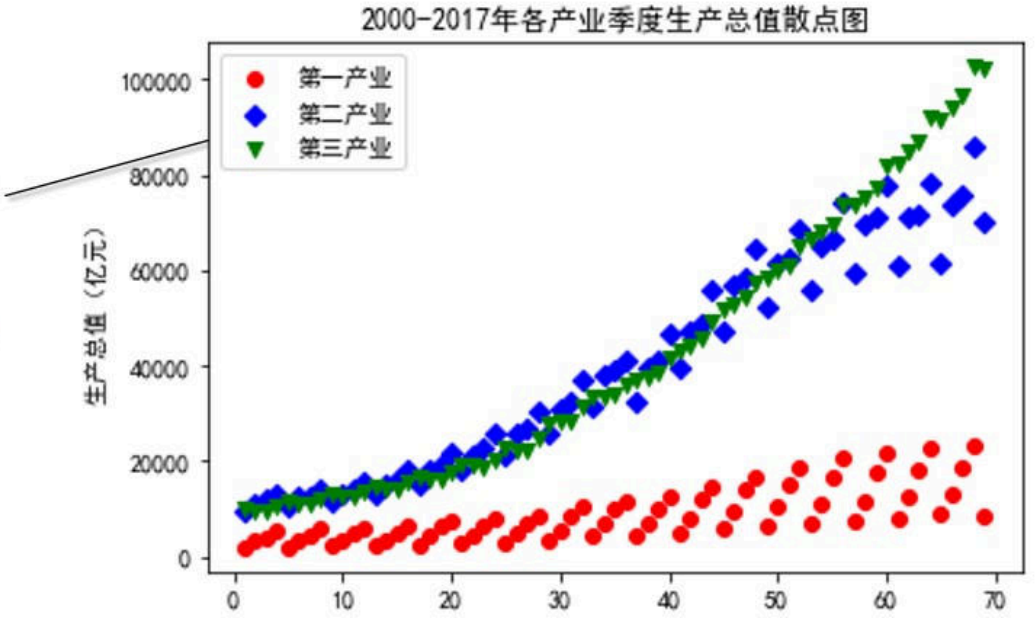
```
## 子图1
ax1 = p.add_subplot(2,1,1)                                  #创建2行1列的子图，并绘制第1幅
plt.scatter(values[:,0],values[:,3], marker='o',c='r')      # 绘制散点
plt.scatter(values[:,0],values[:,4], marker='D',c='b')      # 绘制散点
plt.scatter(values[:,0],values[:,5], marker='v',c='g')      # 绘制散点
plt.ylabel('生产总值（亿元）')                                # 添加纵轴标签
plt.title('2000-2017年各产业季度生产总值散点图')               # 添加图表标题
plt.legend(['第一产业','第二产业','第三产业'])                 # 添加图例
```

78

# 任务实现

## 1. 绘制2000-2017各产业与行业的国民生产总值散点图

- 通过这张散点图看出，我国现阶段国民经济增长的主要动力是第三产业，其次是第二产业。



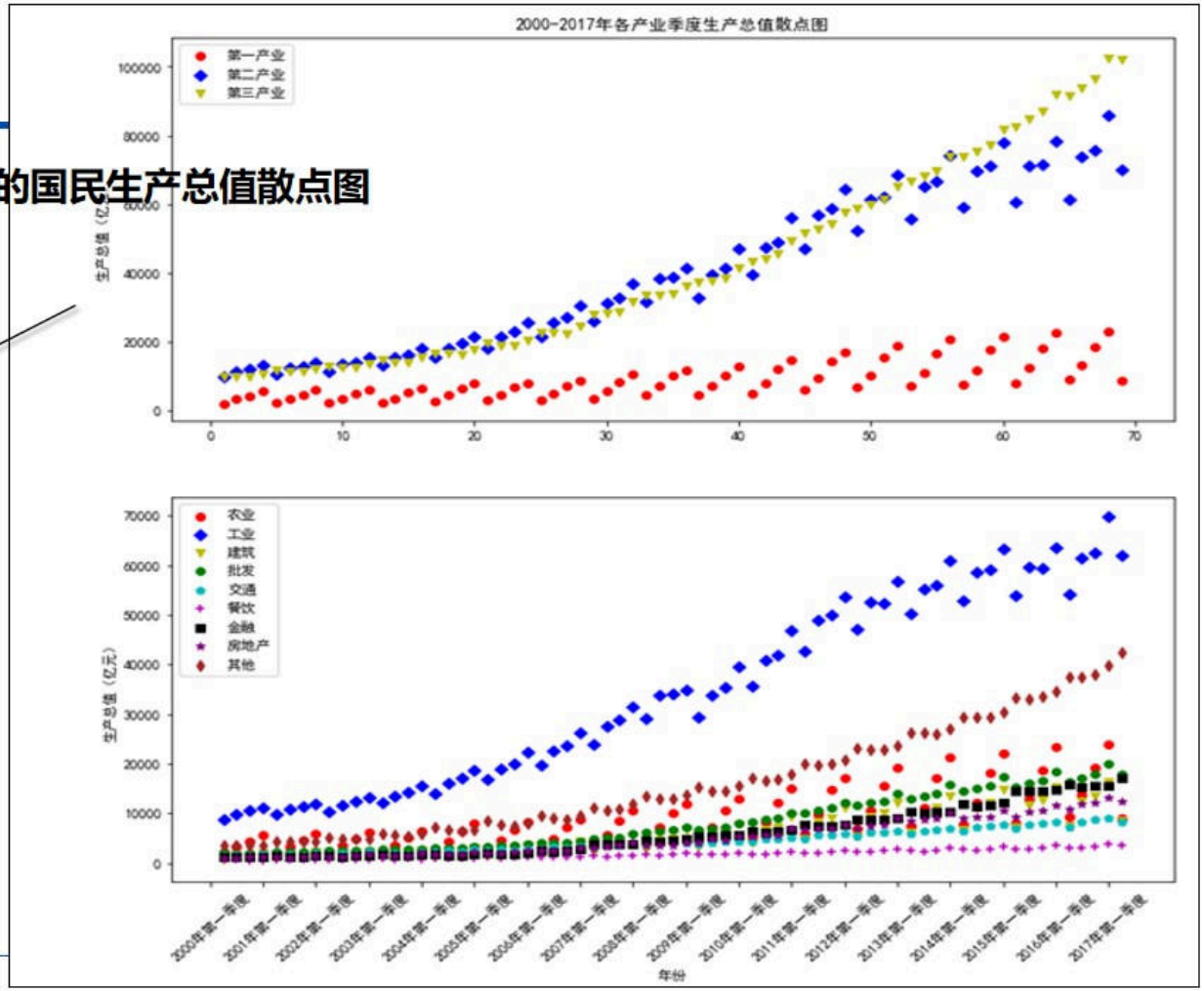
# 任务实现

## 1. 绘制2000-2017各产业与行业的国民生产总值散点图

```
ax2 = p.add_subplot(2,1,2)                    ## 子图2
plt.scatter(values[:,0], values[:,6], marker='o', c='r')
plt.scatter(values[:,0], values[:,7], marker='D', c='b')
plt.scatter(values[:,0], values[:,8], marker='v', c='y')
plt.scatter(values[:,0], values[:,9], marker='8', c='g')
plt.scatter(values[:,0], values[:,10], marker='p', c='c')
plt.scatter(values[:,0], values[:,11], marker='+', c='m')
plt.scatter(values[:,0], values[:,12], marker='s', c='k')
plt.scatter(values[:,0], values[:,13], marker='*', c='purple')
plt.scatter(values[:,0], values[:,14], marker='d', c='brown')
plt.legend(['农业','工业','建筑','批发','交通', '餐饮','金融','房地产','其他'])
plt.xlabel('年份')
plt.ylabel('生产总值（亿元）')
plt.xticks(range(0,70,4), values[range(0,70,4),1], rotation=45)
plt.show()
```

# 任务实现

## 1.绘制2000-2017各产业与行业的国民生产总值散点图




- 通过这个散点图可以看出，我国现阶段国民经济增长的主要动力是第三产业，其次是第二产业。
- 从行业来看，工业、其他行业和农业对整体国民经济贡献度最大。

# 任务实现

## 2. 绘制2000-2017各产业与行业的国民生产总值折线图

- 通过绘制2000-2017各产业与行业的国民生产总值折线图，分别能够发现我国经济各产业与各行业增长趋势。

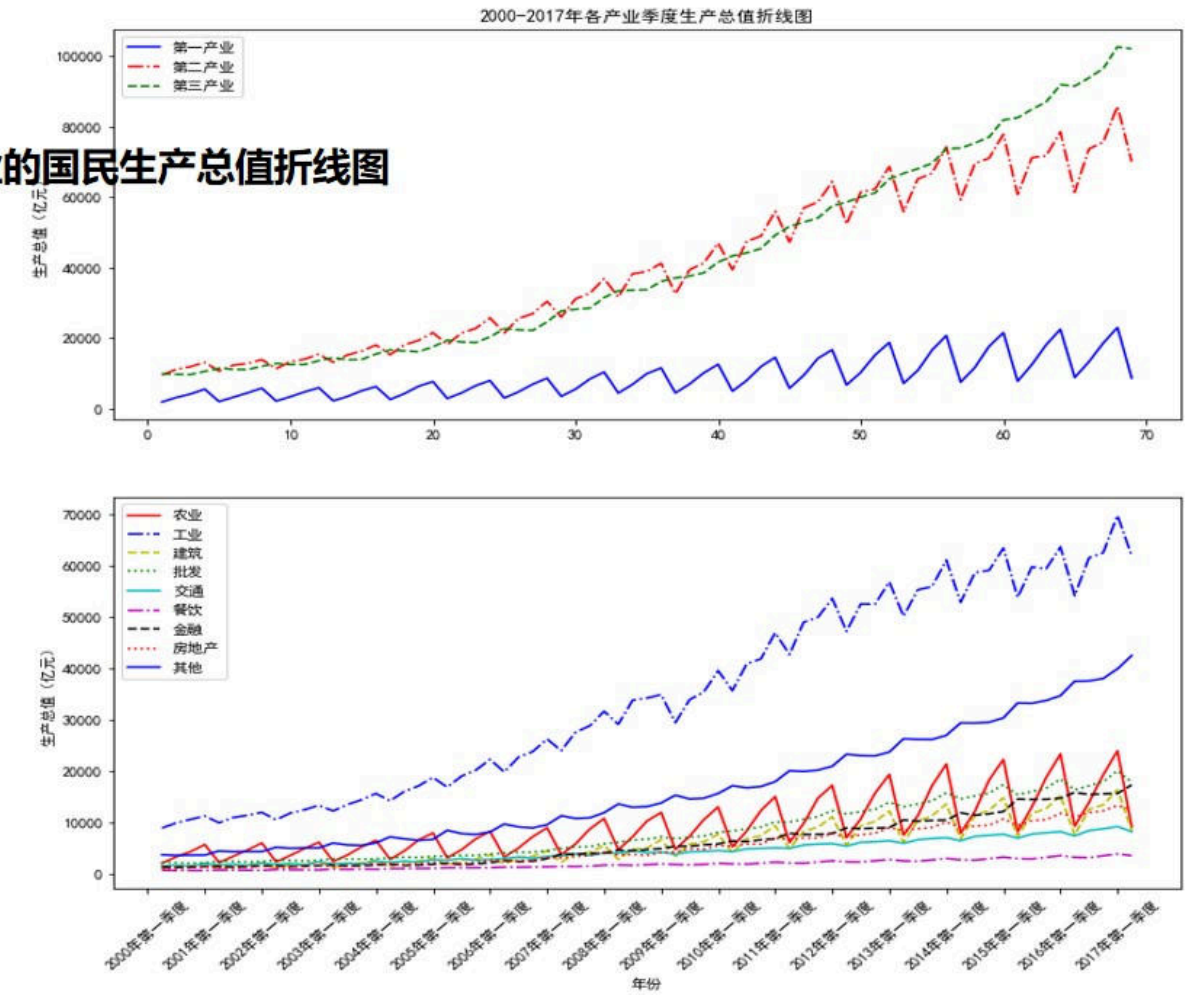




雨课堂 Rain Classroom

# 任务实现

## 2. 绘制2000-2017各产业与行业的国民生产总值折线图



# 任务实现

## 2. 绘制2000-2017各产业与行业的国民生产总值折线图

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = 'SimHei'          # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False          # 设置负号显示
data = np.load('../data/国民经济核算季度数据.npz')
name = data['columns']                              # 提取其中的columns数组，视为数据的标签
values = data['values']                             # 提取其中的values数组，数据的存在位置
p = plt.figure(figsize=(8,7))                       #设置画布
```





雨课堂 Rain Classroom

# 任务实现

**2. 绘制2000-2017各产业与行业的国民生产总值折线图**

```
## 子图1
ax3 = p1.add_subplot(2,1,1)                    #创建2行1列的子图，并绘制第1幅
plt.plot(values[:,0], values[:,3], 'b-',
         values[:,0], values[:,4], 'r-.',
         values[:,0], values[:,5], 'g--')      # 绘制折线图
plt.ylabel('生产总值（亿元）')                  # 添加纵轴标签
plt.title('2000-2017年各产业季度生产总值折线图')# 添加图表标题
plt.legend(['第一产业','第二产业','第三产业'])  # 添加图例
```



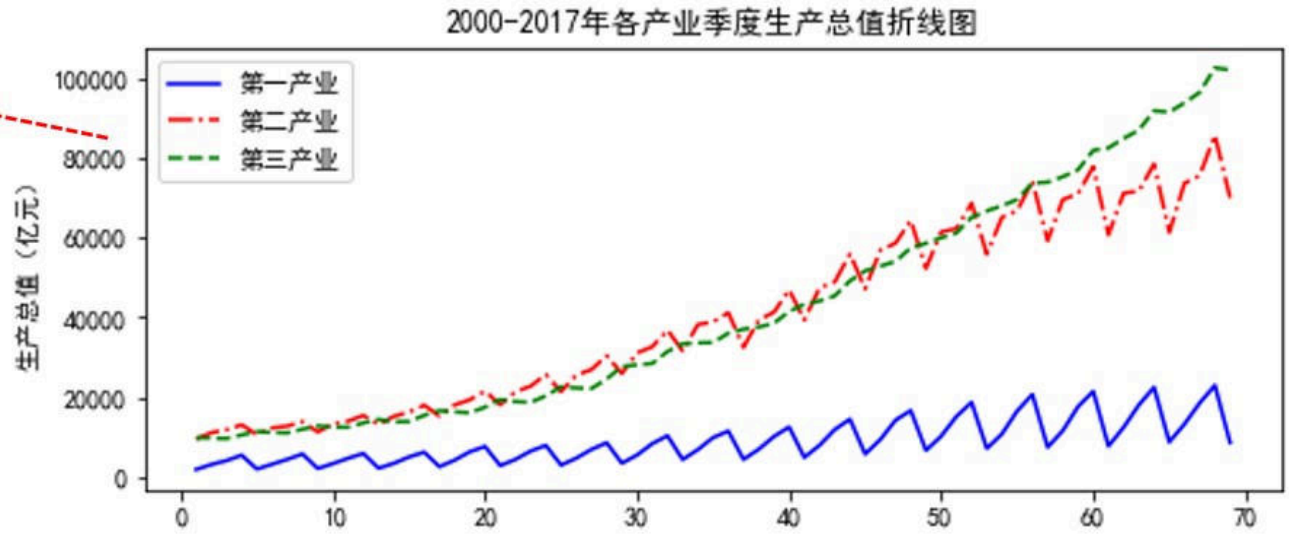


雨课堂 Rain Classroom

# 任务实现

## 2. 绘制2000-2017各产业与行业的国民生产总值折线图

- 通过折线图看出，我国整体经济呈现增长趋势，其中第一产业增长相对较慢，但是周期性最明显



# 任务实现

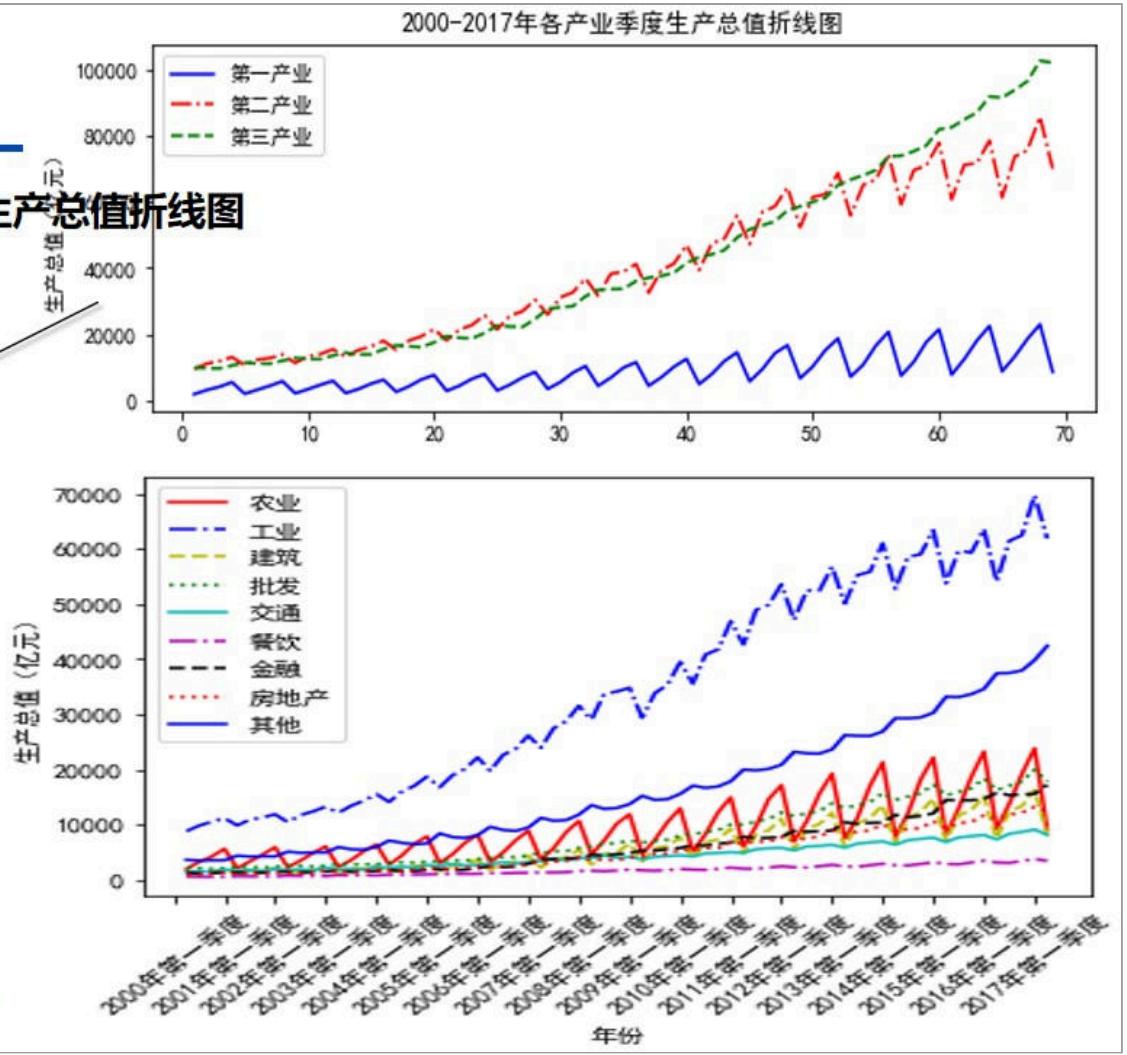
**2. 绘制2000-2017各产业与行业的国民生产总值折线图**

```
## 子图2
ax4 = p1.add_subplot(2,1,2)
plt.plot(values[:,0],values[:,6], 'r-', values[:,0],values[:,7], 'b-.', values[:,0],values[:,8],'y--',
         values[:,0],values[:,9], 'g:', values[:,0],values[:,10], 'c-', values[:,0],values[:,11], 'm-.',
         values[:,0],values[:,12], 'k--', values[:,0],values[:,13], 'r:', values[:,0],values[:,14], 'b-')
                                                                    # 绘制折线图
plt.legend(['农业','工业','建筑','批发','交通','餐饮','金融','房地产','其他'])   # 添加图例
plt.xlabel('年份')                                                  # 添加横轴标签
plt.ylabel('生产总值（亿元）')                                       # 添加纵轴标签
plt.xticks(range(0,70,4),values[range(0,70,4),1],rotation=45)       # 添加x轴刻度数目与取值
plt.savefig('../tmp/2000-2017年季度各行业生产总值折线子图.png')
plt.show()
```

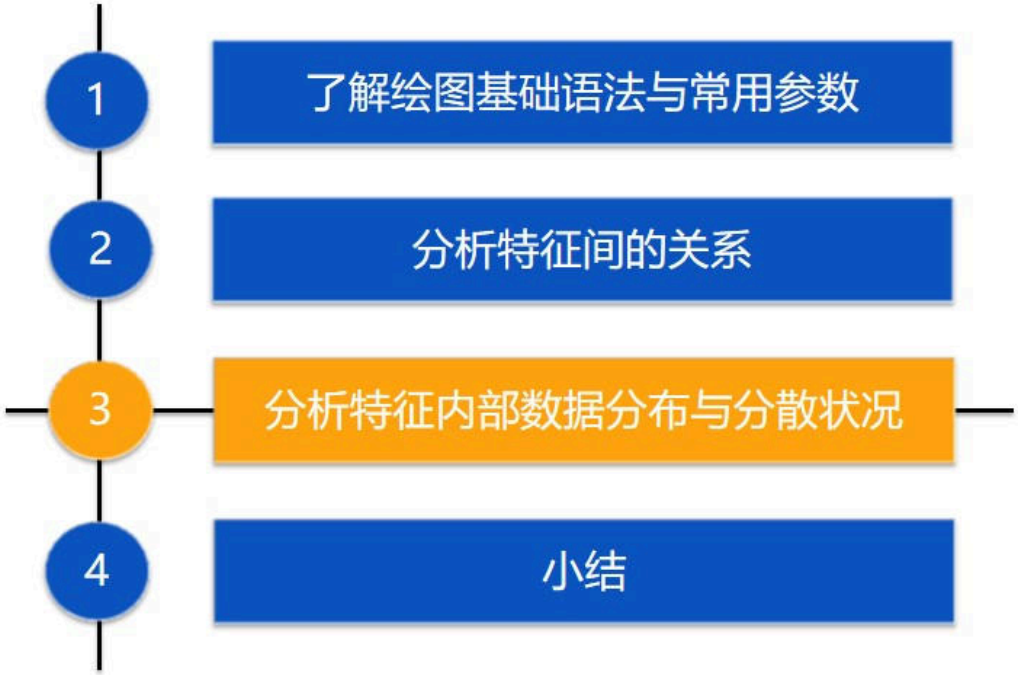
# 任务实现

## 2. 绘制2000-2017各产业与行业的国民生产总值折线图

- 通过折线图看出，我国整体经济呈现增长趋势，其中第一产业增长相对较慢，但是周期性最明显
- 农业的周期性和第一产业的周期性基本吻合。
- 工业和第二产业的增长趋势基本一致。
- 同时，除了餐饮行业外，其他行业均呈现较为明显的增长趋势。

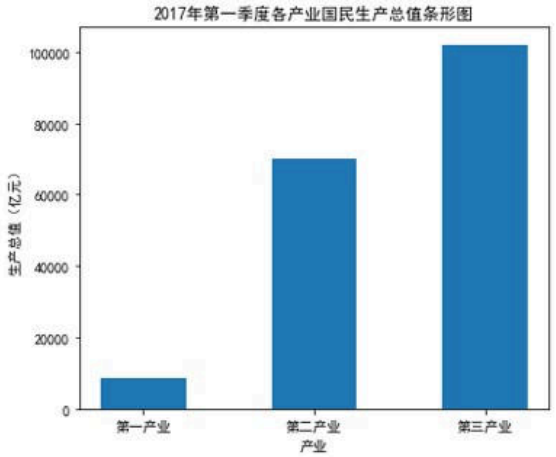


# 目录

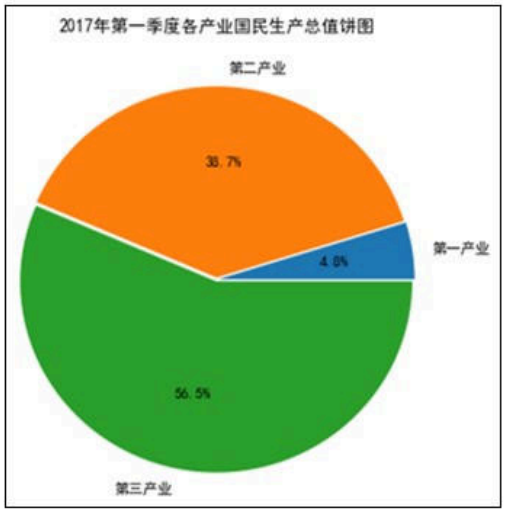


# 分析特征内部数据分布与分散状况

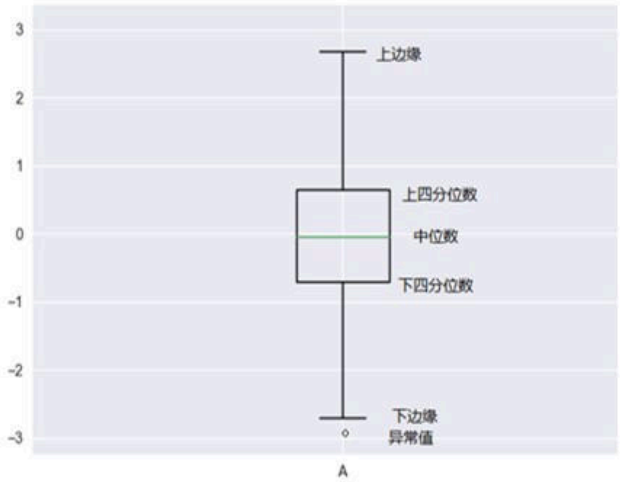
## 条形图




## 饼图




## 箱线图

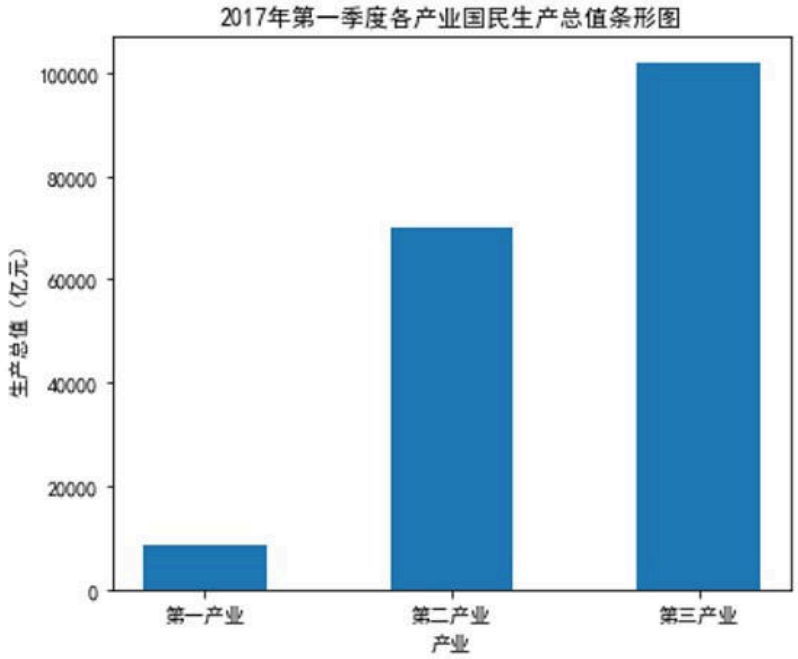



- 条形图、饼图、箱线图是另外3种数据分析常用的图形，主要用于分析数据内部分布状态和分散状态。

# 绘制条形图

## 条形图

- 条形图（bar chart），是用宽度相同的条形的高度或长短来表示数据多少的图形。
- 条形图可以横置或纵置，纵置时也称为柱形图（column chart）。
- 此外，条形图有简单条形图、复式条形图等形式。



# 绘制条形图

## bar函数

*matplotlib.pyplot.bar( left, height, width = 0.8, bottom = None, hold = None, data = None, ** kwargs )*

- 常用参数及说明如下表所示：

| 参数名称 | 说明 |
| --- | --- |
| left | 接收array。表示x轴数据。无默认。 |
| height | 接收array。表示x轴所代表数据的数量。无默认。 |
| width | 接收0-1之间的float。指定条形图宽度。默认为0.8。 |
| color | 接收特定string或者包含颜色字符串的array。表示条形图颜色。默认为None。 |





雨课堂 Rain Classroom

# 绘制条形图

**示例：绘制2017年第一季度各产业国民生产总值条形图（柱状图）**

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = 'SimHei'                  # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False                  # 设置负号显示
data = np.load('data/国民经济核算季度数据.npz')              # 读取数据
name = data['columns']                                      # 提取其中的columns数组，视为数据的标签
values = data['values']                                     # 提取其中的values数组，数据的存在位置
plt.figure(figsize=(6,5))                                   # 设置画布
```





雨课堂 Rain Classroom

# 绘制条形图

**示例：绘制2017年第一季度各产业国民生产总值条形图（柱状图）**

```
plt.bar(range(3),values[-1,3:6],width = 0.5)          # 绘制柱状图
plt.xlabel('产业')                                    # 添加x轴标签
plt.ylabel('生产总值（亿元）')                        # 添加y轴名称
label = ['第一产业','第二产业','第三产业']            # 刻度标签
plt.xticks(range(3),label)                            # 添加x轴刻度数目与取值
plt.title('2017年第一季度各产业国民生产总值柱状图')   # 添加图表标题
plt.savefig('../tmp/2017年第一季度各产业国民生产总值柱状图.png')
plt.show()
```



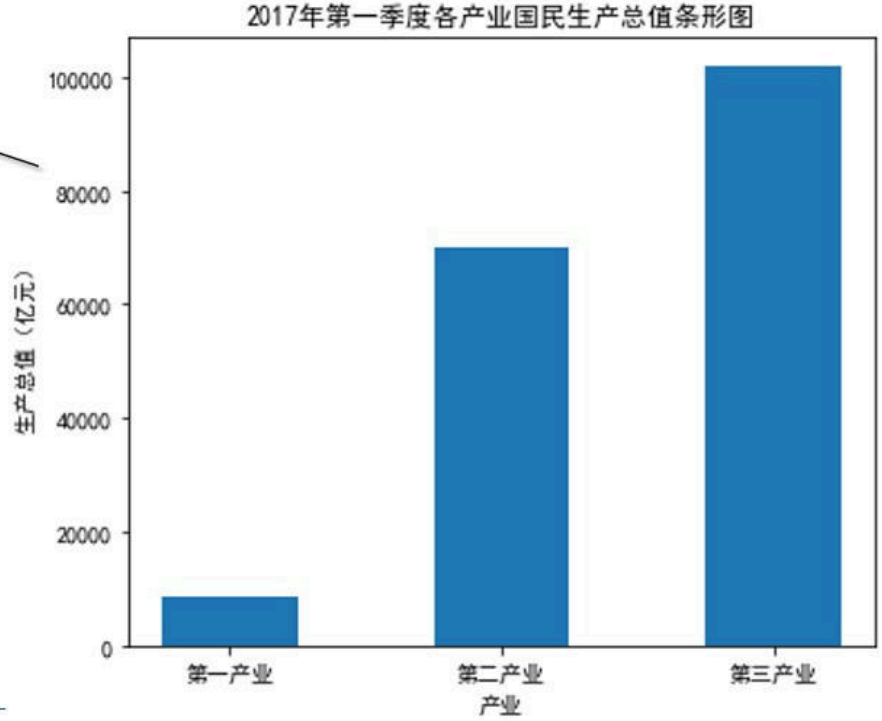


雨课堂 Rain Classroom

# 绘制条形图

**示例：绘制2017年第一季度各产业国民生产总值条形图（柱状图）**

- 从条形图看出，2017年第一季度的第一产业生产总值不到第二产业的六分之一，基本与第三产业的十分之一持平。
- 第二产业生产总值和第三产业生产总值相差大约三分之一。



# 绘制条形图

**示例：绘制2017年第一季度各行业国民生产总值条形图**

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = 'SimHei'                  # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False                  # 设置负号显示
data = np.load('data/国民经济核算季度数据.npz')              # 读取数据
name = data['columns']                                      # 提取其中的columns数组，视为数据的标签
values = data['values']                                     # 提取其中的values数组，数据的存在位置
plt.figure(figsize=(6,5))                                   # 设置画布
```





雨课堂 Rain Classroom

# 绘制条形图

**示例：绘制2017年第一季度各行业国民生产总值条形图**

```
plt.barh(range(9),values[-1,6:15],height = 0.5)                    # 绘制条形图
plt.ylabel('行业')                                                  # 添加y轴标签
plt.xlabel('生产总值（亿元）')                                       # 添加x轴名称
label = ['农业','工业','建筑','批发','交通','餐饮','金融','房地产','其他']   # 刻度标签
plt.yticks(range(9),label)                                         # 添加y轴刻度数目与取值
plt.title('2017年第一季度各行业国民生产总值条形图')                   # 添加图表标题
plt.savefig('../tmp/2017年第一季度各行业国民生产总值条形图.png')
plt.show()
```

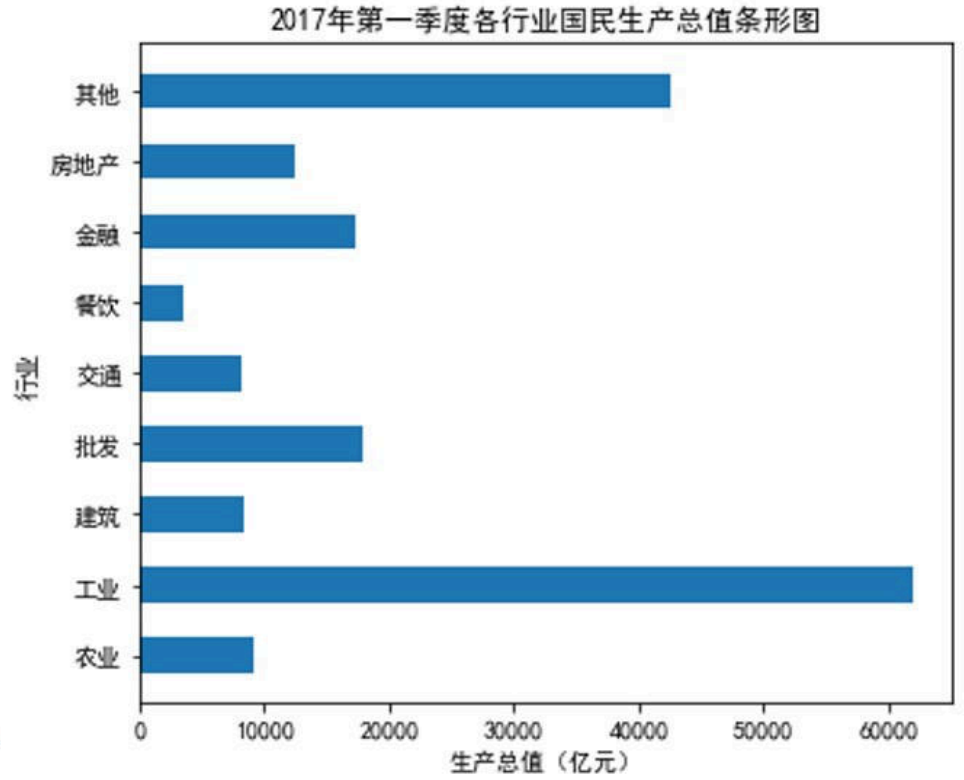




雨课堂 Rain Classroom

# 绘制条形图

**示例：绘制2017年第一季度各行业国民生产总值条形图**



# 绘制条形图

**示例：绘制2016-2017年第一季度各行业国民生产总值对比柱状图**

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = 'SimHei'                  # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False                  # 设置负号显示
data = np.load('data/国民经济核算季度数据.npz')              # 读取数据
name = data['columns']                                      # 提取其中的columns数组，视为数据的标签
values = data['values']                                     # 提取其中的values数组，数据的存在位置
plt.figure(figsize=(6,5))                                   # 设置画布
```





雨课堂 Rain Classroom

# 绘制条形图

**示例：绘制2016-2017年第一季度各行业国民生产总值对比柱状图**

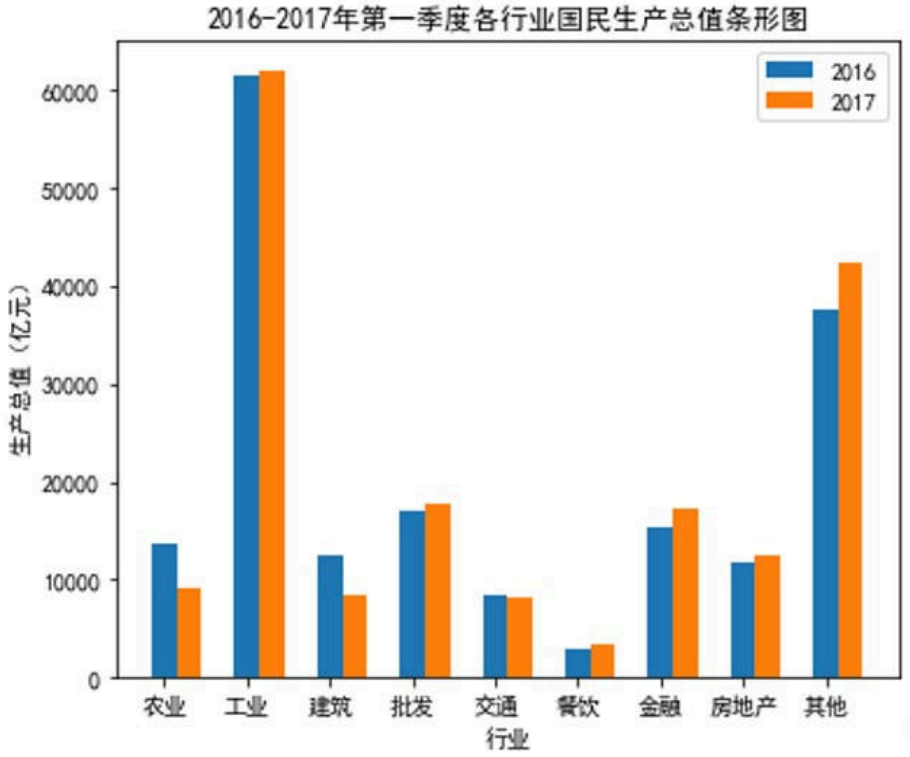
列表推导式

```
x = range(9)
plt.bar(left=x, height=values[-4,6:15], width=0.3, label='2016')                # 绘制柱状图
plt.bar(left=[i+0.3 for i in x],height=values[-1,6:15],width=0.3,label='2017')  # 绘制柱状图
plt.xlabel('行业')                                                               # 添加x轴标签
plt.ylabel('生产总值（亿元）')                                                    # 添加y轴名称
label = ['农业','工业','建筑','批发','交通','餐饮','金融','房地产','其他']       # 刻度标签
plt.xticks(range(9),label)                                                      # 添加x轴刻度数目与取值
plt.title('2016-2017年第一季度各行业国民生产总值对比柱状图')                      # 添加图表标题
plt.legend()                                                                    # 添加图例
plt.savefig('2016-2017年第一季度各行业国民生产总值对比柱状图.png')
plt.show()
```

100

# 绘制条形图

**示例：绘制2016-2017年第一季度各行业国民生产总值对比柱状图**



# 绘制条形图

**示例：绘制2016-2017年第一季度各行业国民生产总值堆叠柱状图**

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = 'SimHei'          # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False          # 设置负号显示
data = np.load('data/国民经济核算季度数据.npz')      # 读取数据
name = data['columns']                              # 提取其中的columns数组，视为数据的标签
values = data['values']                             # 提取其中的values数组，数据的存在位置
plt.figure(figsize=(6,5))                           # 设置画布
```





雨课堂 Rain Classroom

# 绘制条形图

**示例：绘制2016-2017年第一季度各行业国民生产总值堆叠柱状图**

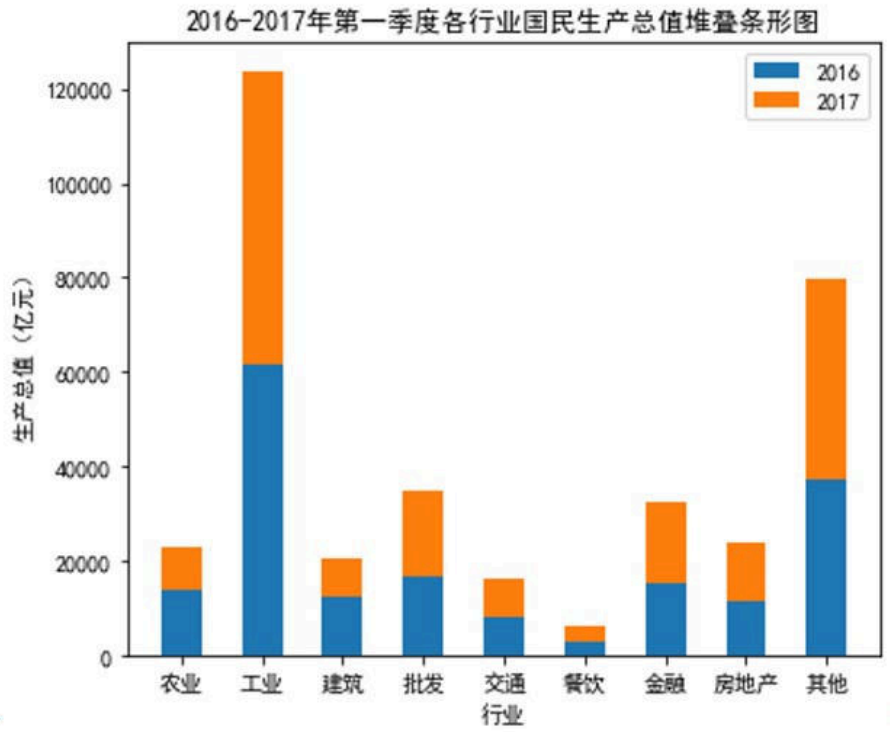
```
x = range(9)
plt.bar(left=x,height=values[-4,6:15],width=0.3,label='2016')      # 绘制柱状图
plt.bar(left=x,height=values[-1,6:15],width=0.3,label='2017', bottom=values[-4,6:15])
plt.xlabel('行业')                                                 # 添加x轴标签
plt.ylabel('生产总值（亿元）')                                      # 添加y轴名称
label = ['农业','工业','建筑','批发','交通','餐饮','金融','房地产','其他']   # 刻度标签
plt.xticks(range(9), label)                                        # 添加x轴刻度数目与取值
plt.title('2016-2017年第一季度各行业国民生产总值堆叠柱状图')          # 添加图表标题
plt.legend()                                                       # 添加图例
plt.savefig('2016-2017年第一季度各行业国民生产总值堆叠柱状图.png')
plt.show()
```

103


雨课堂 Rain Classroom

# 绘制条形图

**示例：绘制2016-2017年第一季度各行业国民生产总值堆叠柱状图**

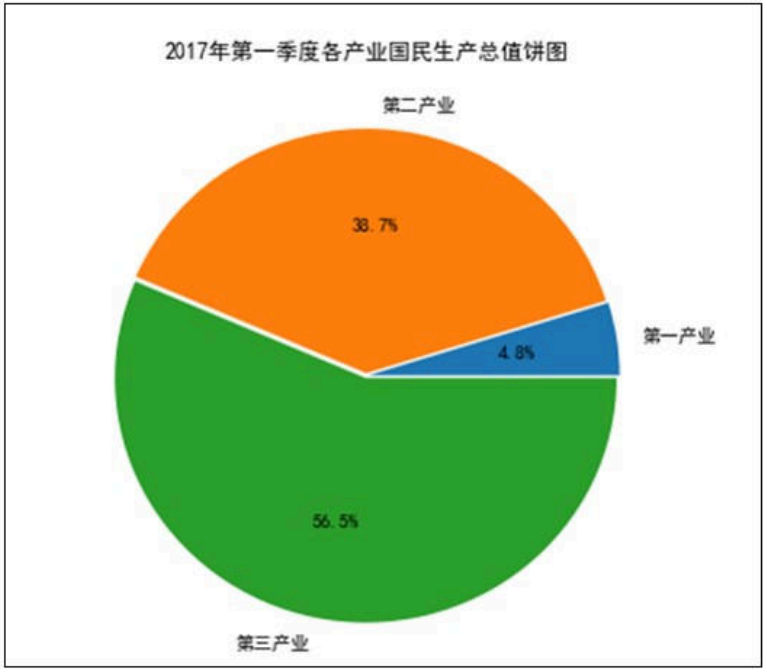


# 绘制饼图

## 饼图

- 饼图（Pie Graph）是将各项的大小与各项总和的比例显示在一张“饼”中，以“饼”的大小来确定每一项的占比。

- 饼图可以比较清楚地反映出部分与部分、部分与整体之间的比例关系，易于显示每组数据相对于总数的大小，而且显现方式直观。



# 绘制饼图

## pie函数

*matplotlib.pyplot.pie(x, explode=None, labels=None, colors=None, autopct=None, pctdistance=0.6, shadow=False, labeldistance=1.1, startangle=None, radius=None, … )*

- 常用参数及说明如下表所示：

| 参数名称 | 说明 | 参数名称 | 说明 |
|---|---|---|---|
| x | 接收array。表示用于绘制饼图的数据。无默认。 | autopct | 接收特定string。指定数值的显示方式。默认为None。 |
| explode | 接收array。表示指定项离饼图圆心为n个半径。默认为None。 | pctdistance | 接收float。指定每一项的比例和距离饼图圆心n个半径。默认为0.6。 |
| labels | 接收array。指定每一项的名称。默认为None。 | labeldistance | 接收float。指定每一项的名称和距离饼图圆心多少个半径。默认为1.1。 |
| color | 接收特定string或者包含颜色字符串的array。表示饼图颜色。默认为None。 | radius | 接收float。表示饼图的半径。默认为1。 |

# 绘制饼图

**绘制2017年第一季度各产业国民生产总值饼图**

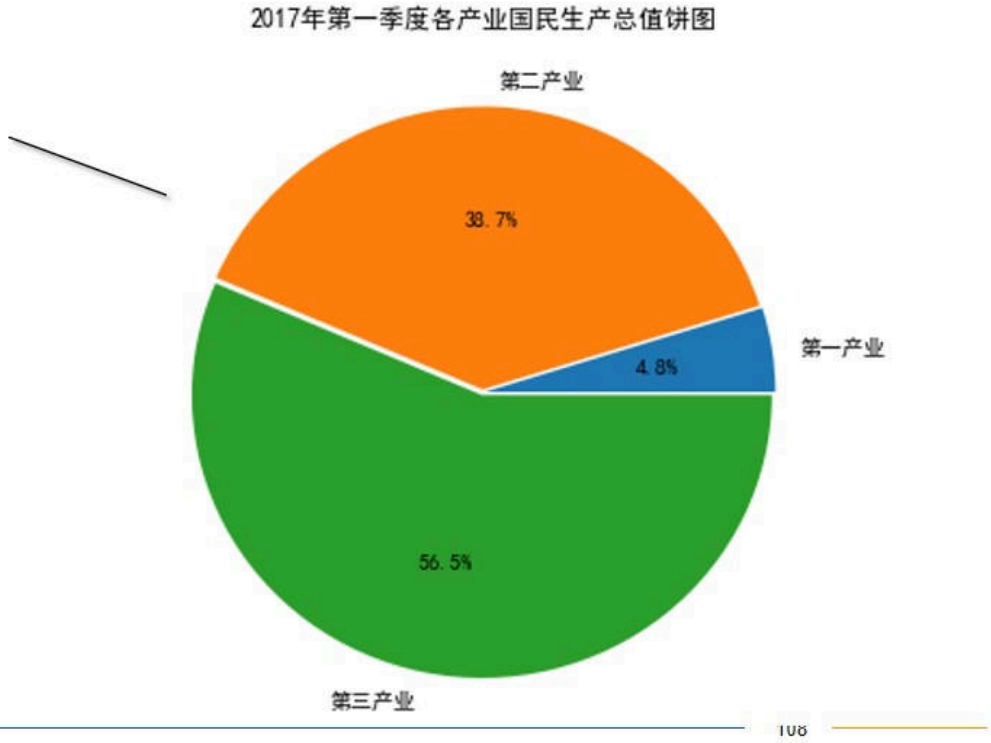
```
plt.figure(figsize=(6,6))                    # 将画布设定为正方形，则绘制的饼图是正圆
label= ['第一产业','第二产业','第三产业']      # 定义饼状图的标签，标签是列表
explode = [0.01,0.01,0.01]                   # 设定各项离心n个半径
## 绘制饼图
plt.pie(values[-1,3:6],explode=explode,labels=label, autopct='%1.1f%%')
plt.title('2017年第一季度各产业国民生产总值饼图')
plt.savefig('../tmp/2017年第一季度各产业生产总值占比饼图')
plt.show()
```

107

# 绘制饼图

## 绘制2017年第一季度各产业国民生产总值饼图

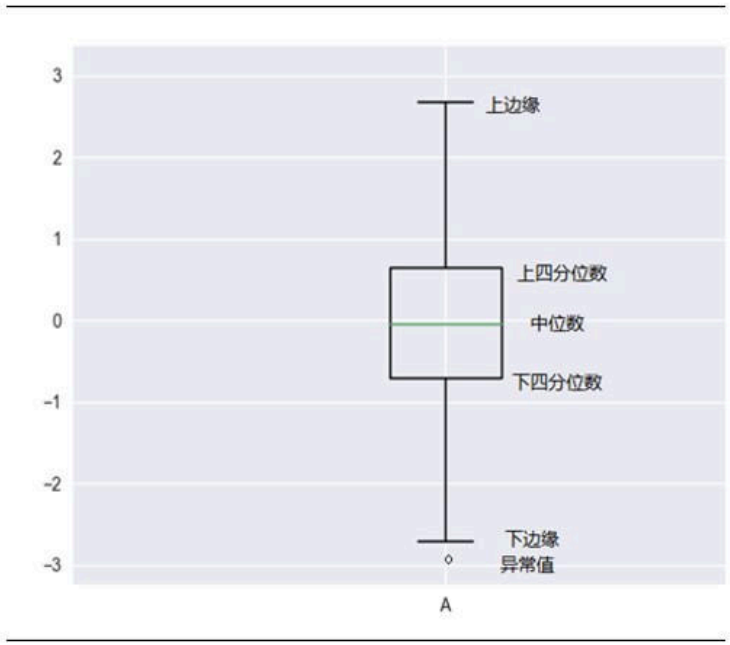
- 通过饼图可以明确看出3个产业在整个国民生产总值中的占比。
- 第一产业不到5%，第三产业超过50%
- 说明现阶段我国经济的主要贡献产业为第三产业。



# 绘制箱线图

## 箱线图

- 箱线图（boxplot）也称箱须图，其绘制需使用常用的统计量，能提供有关数据位置和分散情况的关键信息，尤其在比较不同特征时，更可表现其分散程度差异。
- 箱线图利用数据中的五个统计量（最小值、下四分位数、中位数、上四分位数和最大值）来描述数据
- 可以粗略地看出数据是否具有对称性、分布的分散程度等信息，特别可以用于对几个样本的比较。



# 绘制箱线图

## boxplot函数

*matplotlib.pyplot.boxplot(x, notch=None, sym=None, vert=None, whis=None, positions=None, widths=None, patch_artist=None,meanline=None, labels=None, … )*

- 常用参数及说明如下表所示：

| 参数名称 | 说明 | 参数名称 | 说明 |
|---|---|---|---|
| x | 接收array。表示用于绘制箱线图的数据。无默认。 | positions | 接收array。表示图形位置。默认为None。 |
| notch | 接收boolean。表示中间箱体是否有缺口。默认为None。 | widths | 接收scalar或者array。表示每个箱体的宽度。默认为None。 |
| sym | 接收特定sting。指定异常点形状。默认为None。 | labels | 接收array。指定每一个箱线图的标签。默认为None。 |
| vert | 接收boolean。表示图形是纵向或者横向。默认为None。 | meanline | 接收boolean。表示是否显示均值线。默认为False。 |

# 绘制箱线图

## 绘制2000-2017年各产业国民生产总值箱线图

```
label= ['第一产业','第二产业','第三产业']                    # 定义标签
gdp = (list(values[:,3]), list(values[:,4]), list(values[:,5]))    # 确定数据
plt.figure(figsize=(6,4))
plt.boxplot(gdp,notch=True,labels = label, meanline=True)    # 绘制箱线图
plt.title('2000-2017各产业国民生产总值箱线图')
plt.savefig('../tmp/2000-2017各产业国民生产总值箱线图.png')
plt.show()
```



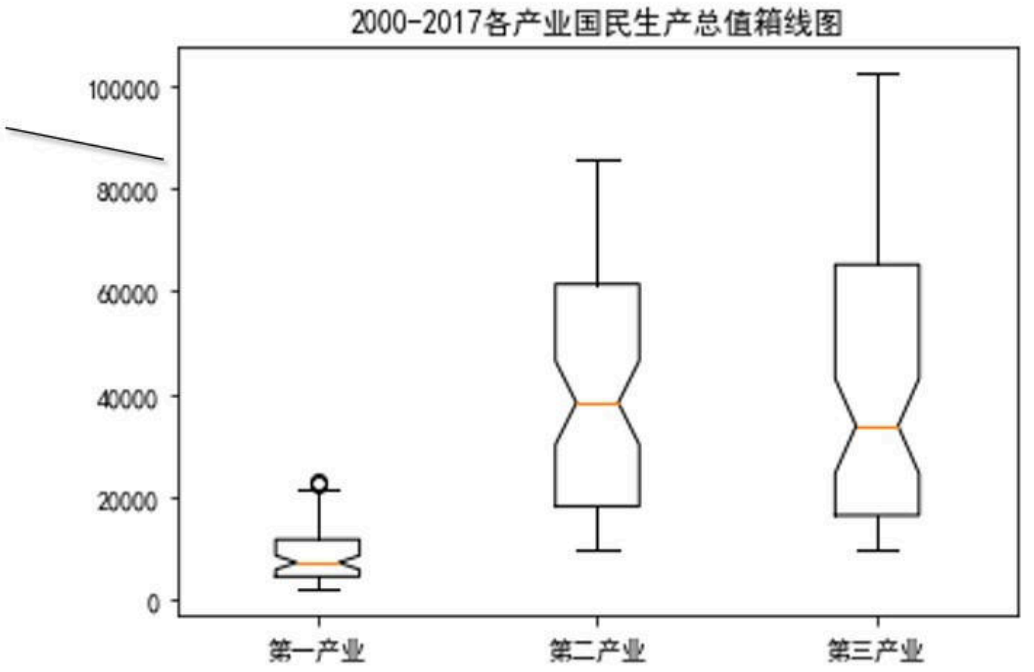


雨课堂 Rain Classroom

# 绘制箱线图

## 绘制2000-2017年各产业国民生产总值箱线图

- 通过这张箱线图看出，在2000-2017年，第一产业在某一年的某个季度具有一个异常值。
- 第三产业整体增速变大，导致了第三产业数据前半部分相对密集，而后半部分相对的分散。



# 任务实现

**1. 绘制国民生产总值构成分布柱状图**

- 通过柱状图分析2000年第一季度和2017年第一季度之间的三大产业的国民生产总值，可以发现各产业绝对数值之间的关系，并通过对比发现产业结构的变化。

- 同理可以得出行业间的绝对数值关系以及17年来行业发展状况。





雨课堂 Rain Classroom

# 任务实现

## 1. 绘制国民生产总值构成分布柱状图

```
import numpy as np
import matplotlib.pyplot as plt
data = np.load('data/国民经济核算季度数据.npz')
name = data['columns']                                  # 提取其中的columns数组，视为数据的标签
values = data['values']                                 # 提取其中的values数组，数据的存在位置
plt.rcParams['font.sans-serif'] = 'SimHei'              # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False
label1 = ['第一产业','第二产业','第三产业']                  # 刻度标签1
label2 = ['农业','工业','建筑','批发','交通','餐饮','金融','房地产','其他']   # 刻度标签2
p = plt.figure(figsize=(12,12))
```



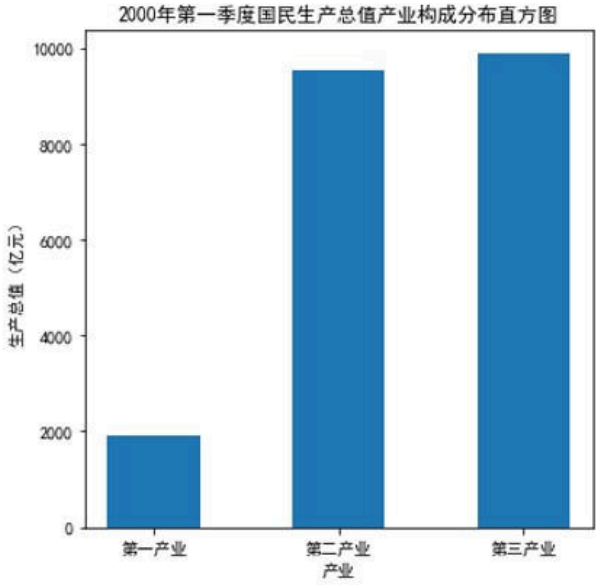


雨课堂 Rain Classroom

# 任务实现

## 1. 绘制国民生产总值构成分布柱状图

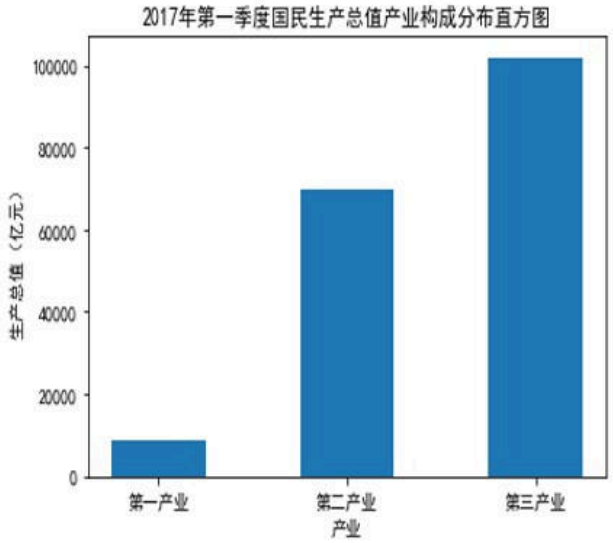
```
## 子图1
ax1 = p.add_subplot(2,2,1)
plt.bar(range(3),values[0,3:6],width = 0.5)      # 绘制柱状图
plt.xlabel('产业')                               # 添加横轴标签
plt.ylabel('生产总值（亿元）')                    # 添加y轴名称
plt.xticks(range(3),label1)
## 添加图表标题
plt.title('2000年第一季度国民生产总值产业构成分布柱状图')
```



# 任务实现

## 1. 绘制国民生产总值构成分布柱状图

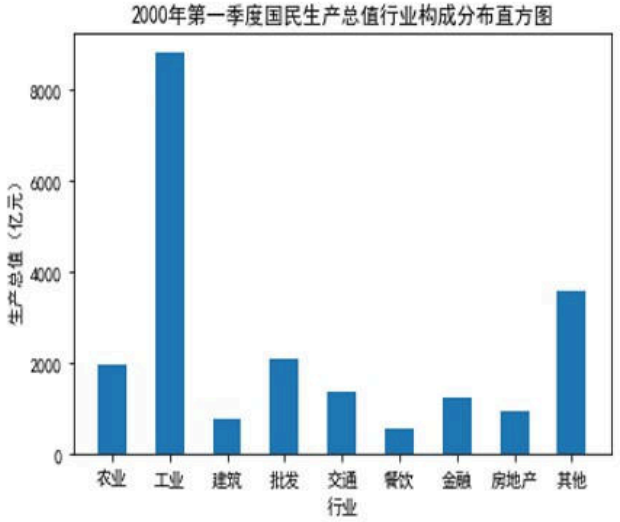
```
## 子图2
ax2 = p.add_subplot(2,2,2)
plt.bar(range(3),values[-1,3:6],width = 0.5)      # 绘制柱状图
plt.xlabel('产业')                                 # 添加x轴标签
plt.ylabel('生产总值（亿元）')                      # 添加y轴名称
plt.xticks(range(3),label1)
## 添加图表标题
plt.title('2017年第一季度国民生产总值产业构成分布柱状图')
```



# 任务实现

## 1. 绘制国民生产总值构成分布柱状图

```
## 子图3
ax3 = p.add_subplot(2,2,3)
plt.bar(range(9),values[0,6:],width = 0.5)    # 绘制柱状图
plt.xlabel('行业')                             # 添加x轴标签
plt.ylabel('生产总值（亿元）')                  # 添加y轴名称
plt.xticks(range(9),label2)
## 添加图表标题
plt.title('2000年第一季度国民生产总值行业构成分布柱状图')
```







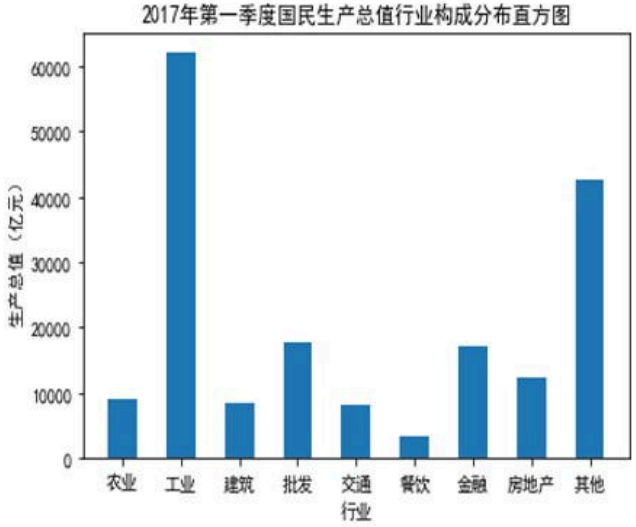

雨课堂 Rain Classroom

# 任务实现

## 1. 绘制国民生产总值构成分布柱状图

```
## 子图4
ax4 = p.add_subplot(2,2,4)
plt.bar(range(9),values[-1,6:],width = 0.5)  # 绘制柱状图
plt.xlabel('行业')                           # 添加x轴标签
plt.ylabel('生产总值（亿元）')                 # 添加y轴名称
plt.xticks(range(9),label2)

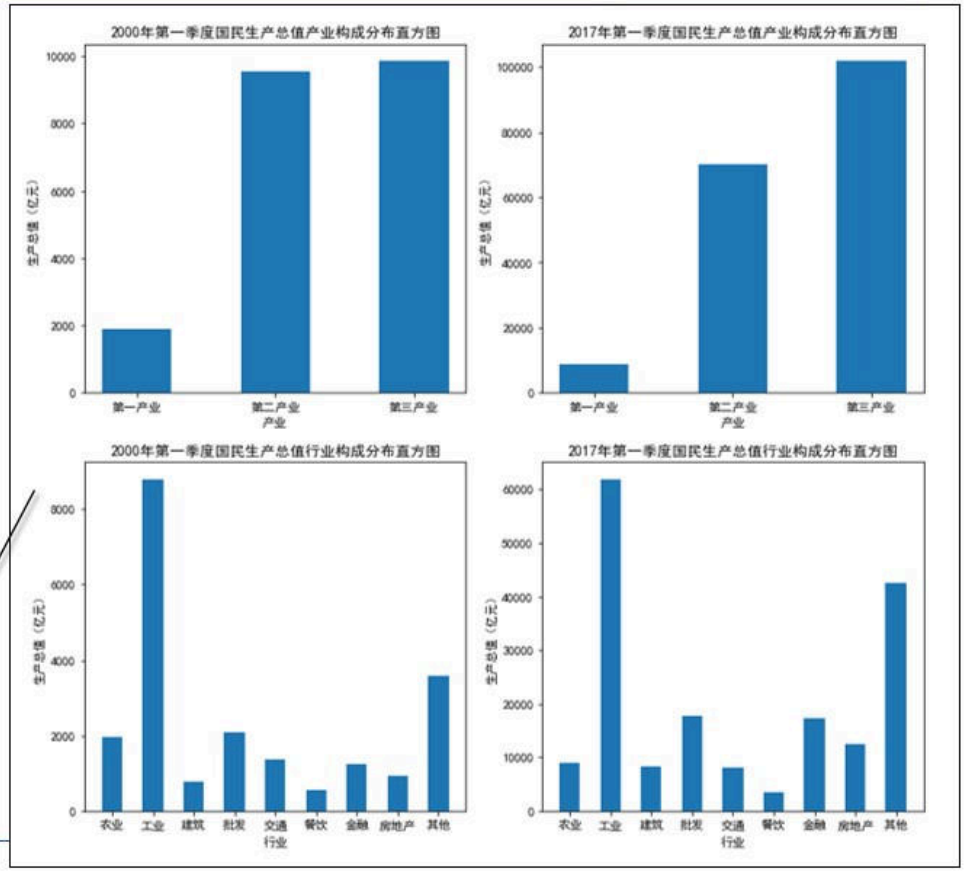
## 添加图表标题
plt.title('2017年第一季度国民生产总值行业构成分布柱状图')
plt.savefig('../tmp/国民生产总值构成分布柱状图.png')
plt.show()
```



# 任务实现

## 1. 绘制国民生产总值构成分布柱状图

- 通过柱状图分析2000年第一季度和2017年第一季度之间的三大产业的国民生产总值，可以发现各产业绝对数值之间的关系，并通过对比发现产业结构的变化。
- 同理可以得出行业间的绝对数值关系以及17年来行业发展状况。




- 通过这几张柱状图对比看出，第一产业与第二产业和第三产业的国民生产总值差距愈发巨大。
- 根据坐标轴变化可以发现，国民生产总值增长近10倍。2010-2017年，金融行业与其他行业增长幅度相比较较为明显。


雨课堂 Rain Classroom

# 任务实现

## 2. 绘制国民生产总值构成分布饼图

- 通过分析2000年与2017年不同的产业和行业在国民生产总值中的占比，可以发现我国产业结构变化和行业变迁。





雨课堂 Rain Classroom

# 任务实现

## 2. 绘制国民生产总值构成分布饼图

```
import numpy as np
import matplotlib.pyplot as plt
data = np.load('data/国民经济核算季度数据.npz')
name = data['columns']                                  # 提取其中的columns数组，视为数据的标签
values = data['values']                                 # 提取其中的values数组，数据的存在位置
plt.rcParams['font.sans-serif'] = 'SimHei'              # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False
label1 = ['第一产业','第二产业','第三产业']                    # 标签1
label2 = ['农业','工业','建筑','批发','交通','餐饮','金融','房地产','其他']   # 标签2
explode1 = [0.01,0.01,0.01]
explode2 = [0.01,0.01,0.01,0.01,0.2,0.01,0.01,0.01,0.01]   # 设定交通子块离圆心远一些
p = plt.figure(figsize=(12,12))
```

121


雨课堂 Rain Classroom

# 任务实现

**2. 绘制国民生产总值构成分布饼图**

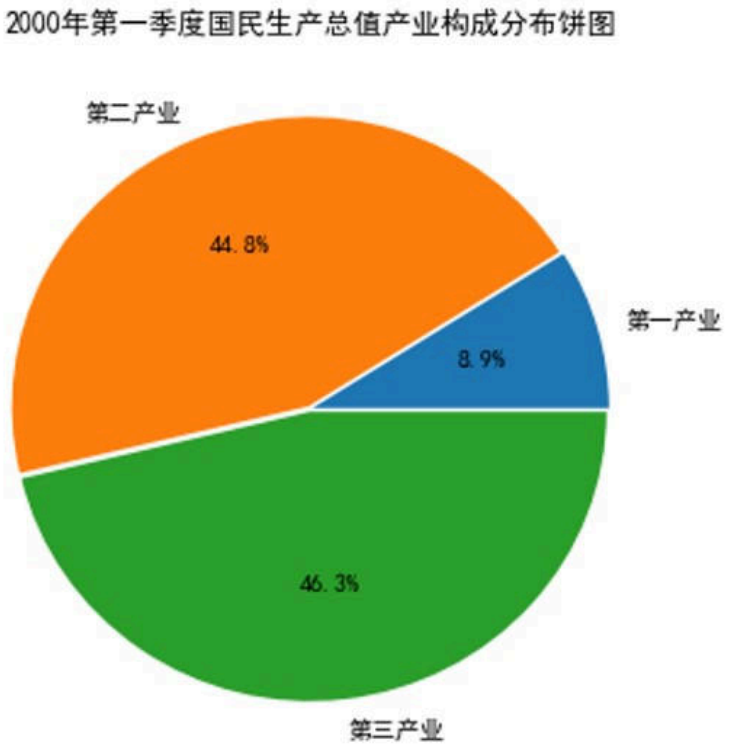
```
## 子图1
ax1 = p.add_subplot(2,2,1)
plt.pie(values[0,3:6],explode=explode1,labels=label1,autopct='%1.1f%%')      # 绘制饼图
plt.title('2000年第一季度国民生产总值产业构成分布饼图')

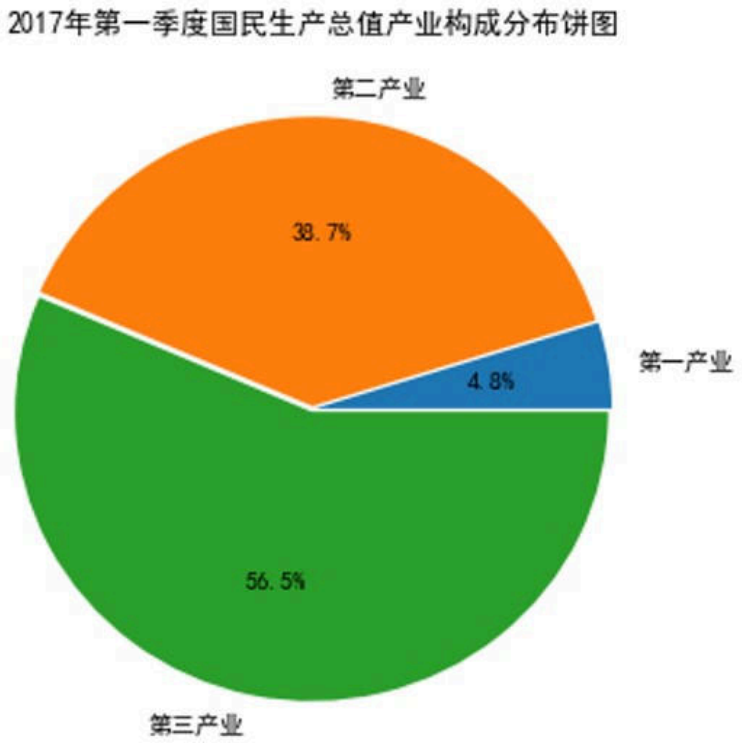
## 子图2
ax2 = p.add_subplot(2,2,2)
plt.pie(values[-1,3:6],explode=explode1,labels=label1,autopct='%1.1f%%')     # 绘制饼图
plt.title('2017年第一季度国民生产总值产业构成分布饼图')
```

122

# 任务实现

## 2. 绘制国民生产总值构成分布饼图



# 任务实现

**2. 绘制国民生产总值构成分布饼图**

```
## 子图3
ax3 = p.add_subplot(2,2,3)
plt.pie(values[0,6:],explode=explode2,labels=label2,autopct='%1.1f%%')   # 绘制饼图
plt.title('2000年第一季度国民生产总值行业构成分布饼图’ )                    # 添加图表标题
## 子图4
ax4 = p.add_subplot(2,2,4)
plt.pie(values[-1,6:],explode=explode2,labels=label2,autopct='%1.1f%%')   # 绘制饼图
plt.title('2017年第一季度国民生产总值行业构成分布饼图’ )                    # 添加图表标题
## 保存并显示图形
plt.savefig('../tmp/国民生产总值构成分布饼图.png')
plt.show()
```

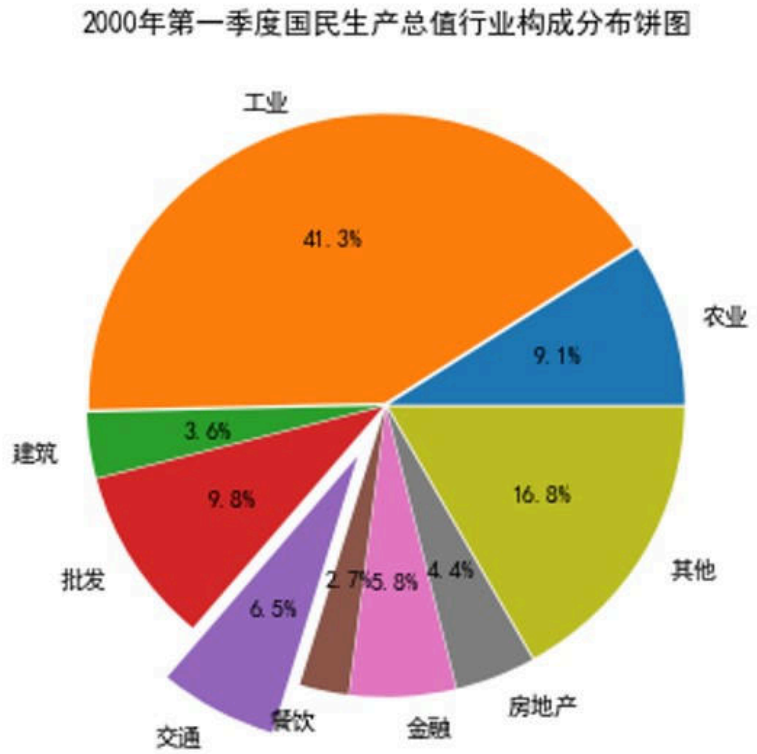


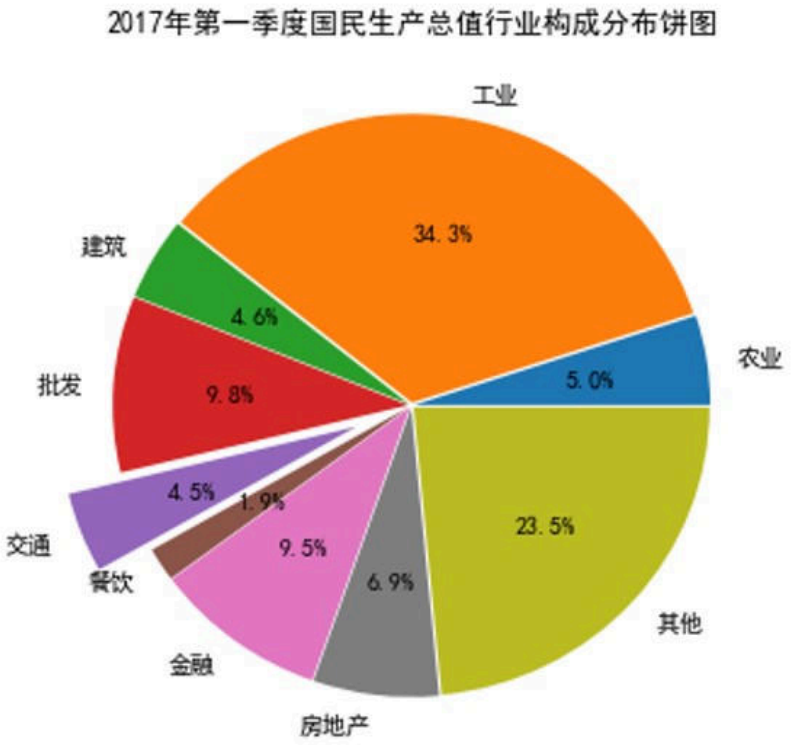


雨课堂 Rain Classroom

# 任务实现

## 2. 绘制国民生产总值构成分布饼图



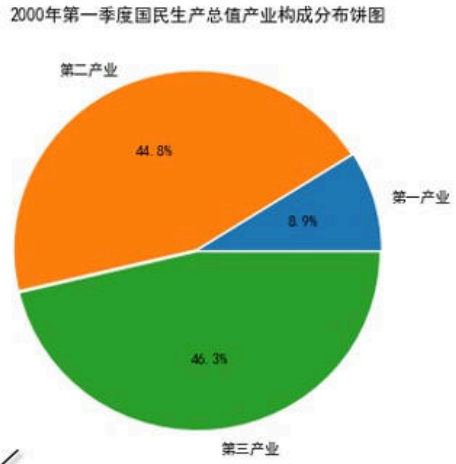



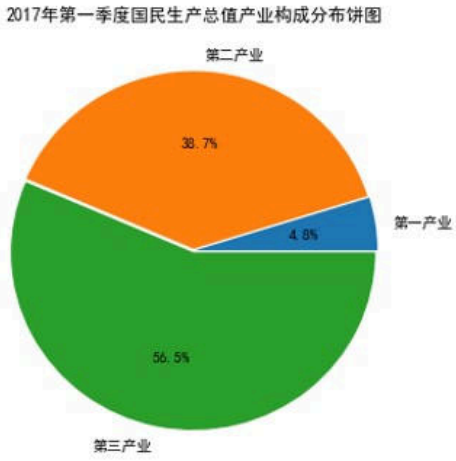
# 任务实现

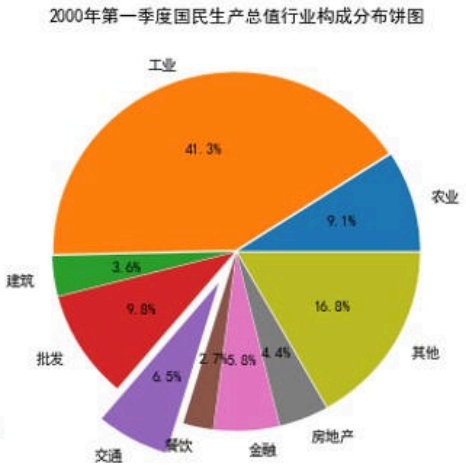
## 2. 绘制国民生产总值构成分布饼图

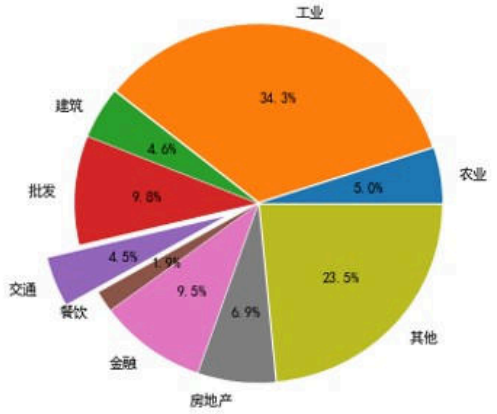
- 通过分析2000年与2017年不同的产业和行业在国民生产总值中的占比，可以发现我国产业结构变化和行业变迁。

- 通过这个饼图可以看出
- 2000-2017年，第三产业在整个国民生产总值中的占比约提高了10%。
- 第一产业和第二产业在国民生产总值中的占比分别下降了约4%和6%。
- 工业在整个国民生产总值中的比例下降了7%，
- 其他行业与金融行业则分别提升了6.7%和3.7%



# 任务实现

### 3. 绘制国民生产总值分散情况箱线图

- 通过分析2000年至2017年不同的产业和行业在国民生产总值中的分散情况，可以发现整体分散情况，从而判断整体增速是否加快。





雨课堂 Rain Classroom

# 任务实现

### 3. 绘制国民生产总值分散情况箱线图

```
import numpy as np
import matplotlib.pyplot as plt

data = np.load('data/国民经济核算季度数据.npz')

name = data['columns']                          # 提取其中的columns数组，视为数据的标签
values = data['values']                         # 提取其中的values数组，数据的存在位置
plt.rcParams['font.sans-serif'] = 'SimHei'      # 设置中文显示
plt.rcParams['axes.unicode_minus'] = False

p = plt.figure(figsize=(8,8))
```





雨课堂 Rain Classroom

# 任务实现

## 3. 绘制国民生产总值分散情况箱线图

```
## 子图1
ax1 = p.add_subplot(2,1,1)
gdp1 = (list(values[:,3]), list(values[:,4]), list(values[:,5]))
label1 = ['第一产业','第二产业','第三产业']                    # 标签1

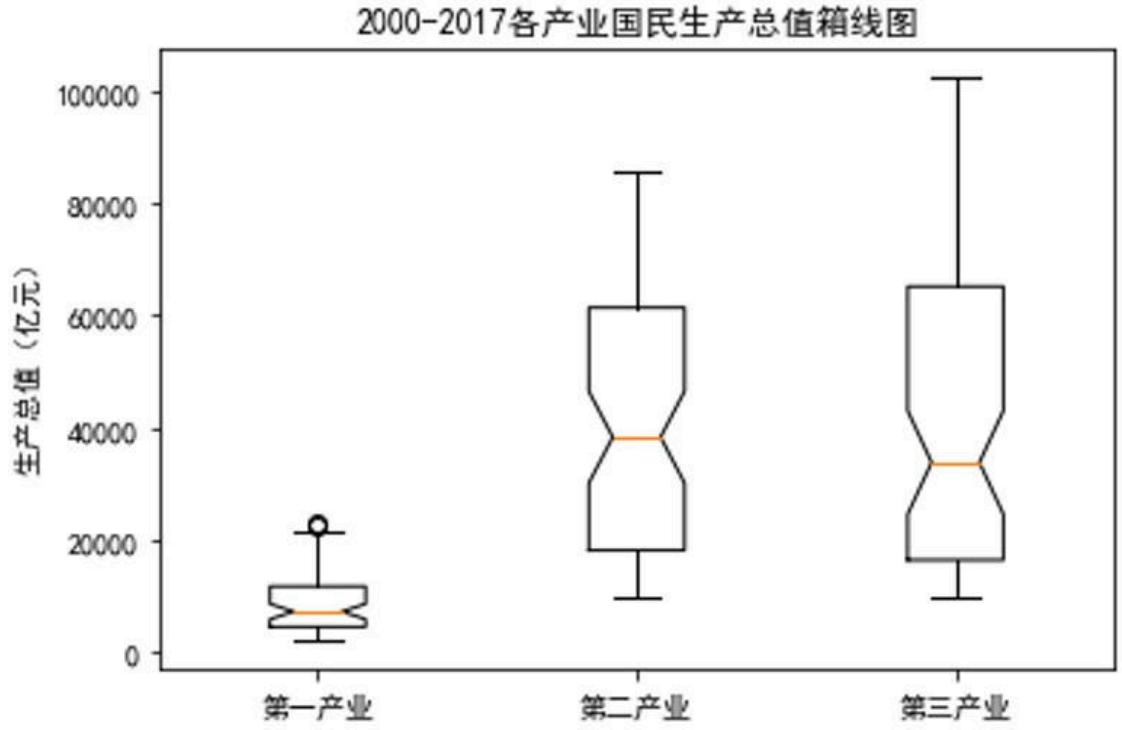
## 绘制箱线图
plt.boxplot(gdp1,notch=True,labels = label1, meanline=True)
plt.title('2000-2017各产业国民生产总值箱线图')
plt.ylabel('生产总值（亿元）')                                 # 添加y轴名称
```



雨课堂 Rain Classroom

# 任务实现

### 3. 绘制国民生产总值分散情况箱线图



# 任务实现

### 3. 绘制国民生产总值分散情况箱线图

```
## 子图2
ax2 = p.add_subplot(2,1,2)
gdp2 = ([list(values[:,i]) for i in range(6,15)])
label2 = ['农业','工业','建筑','批发','交通 ','餐饮','金融','房地产','其他']        # 标签2
## 绘制箱线图
plt.boxplot(gdp2,notch=True,labels = label2, meanline=True)
plt.title('2000-2017各行业国民生产总值箱线图')
plt.xlabel('行业')                                                    # 添加x轴标签
plt.ylabel('生产总值（亿元）')                                          # 添加y轴名称
## 保存并显示图形
plt.savefig('../tmp/国民生产总值分散情况箱线图.png')
plt.show()
```

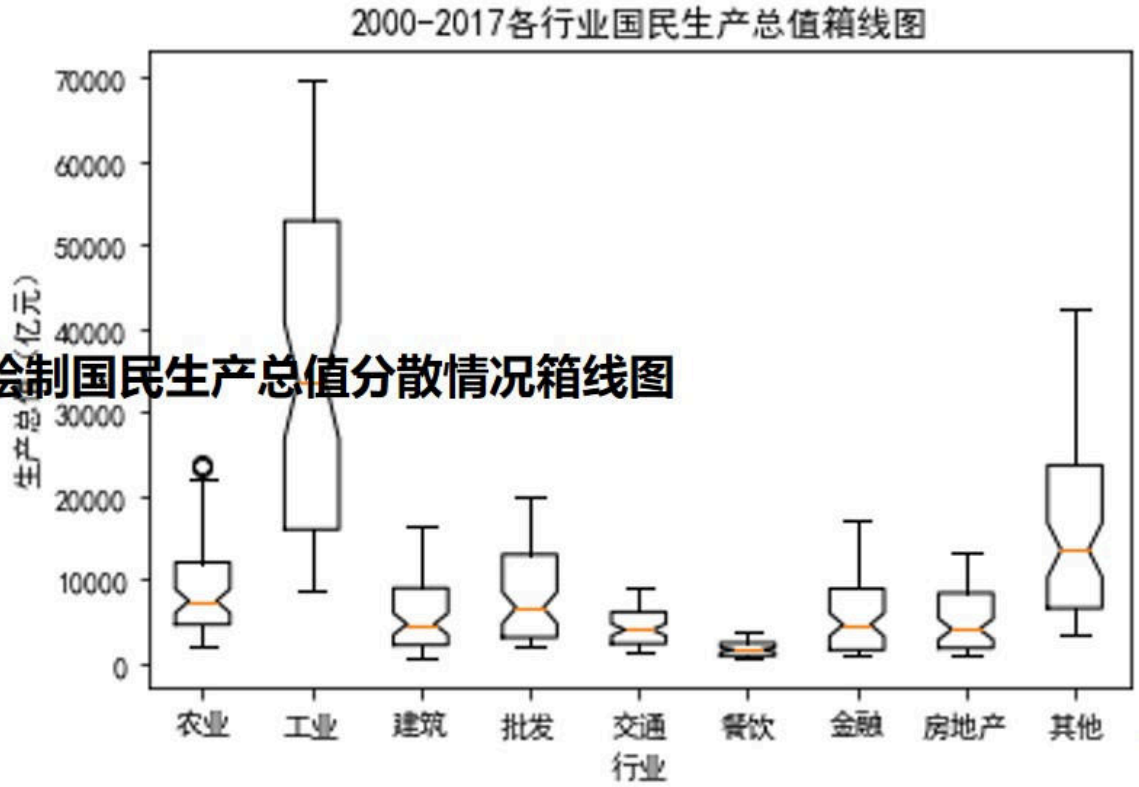




雨课堂 Rain Classroom

# 任务实现

## 3. 绘制国民生产总值分散情况箱线图





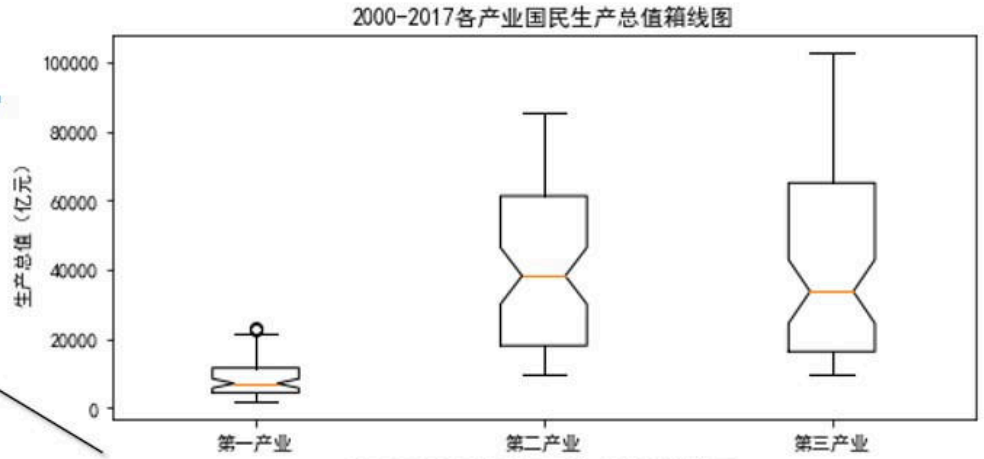


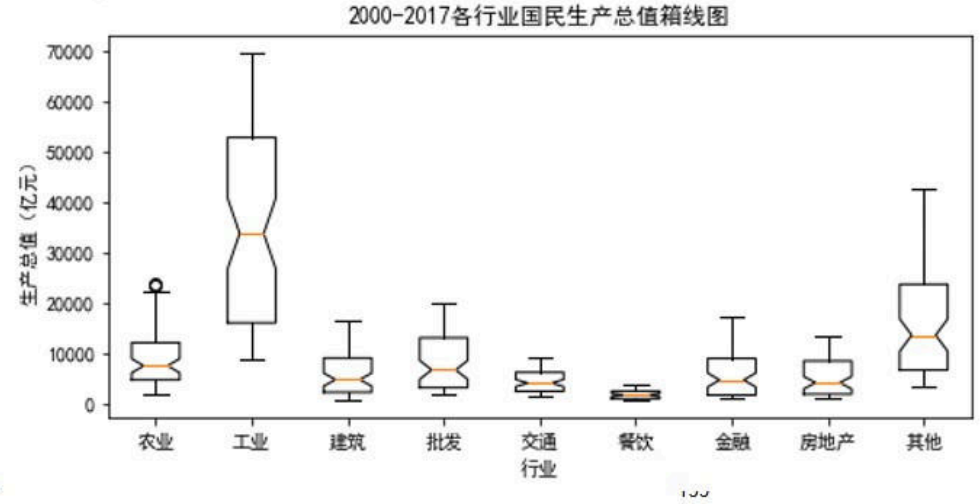

雨课堂 Rain Classroom

# 任务实现

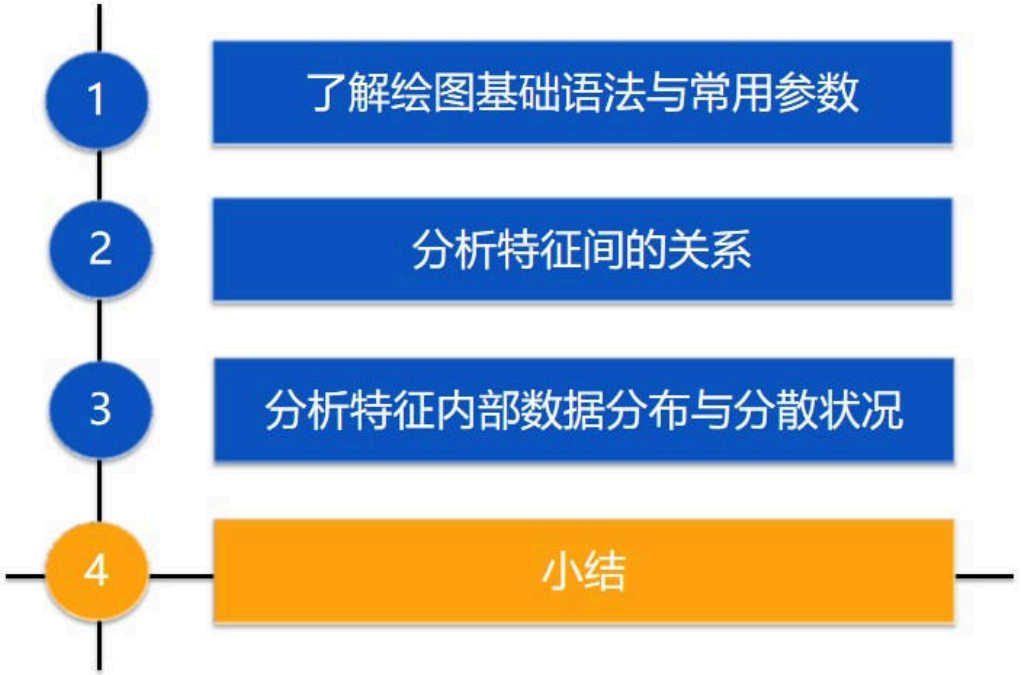
## 3. 绘制国民生产总值分散情况箱线图

- 通过箱线图可以看出：
- 整体经济趋势是上升的。
- 产业中的第二产业增长平缓。
- 行业中的工业与餐饮业的增长比较平缓，其他行业、批发行业、建筑行业、金融行业和房地产行业增速均有所加快



# 目录



# 小结

本章以2000至2017年各季度国民生产总值数据为例，介绍了pyplot绘图的基本语法、常用参数。

- 介绍了分析特征间相关关系的**散点图**。
- 分析特征间趋势关系的**折线图**。
- 分析特征内部数据分布的**条形图**和**饼状图**。
- 以及分析特征内部数据分散情况的**箱线图**。

为读者后续深入学习Matplotlib数据可视化打下了深厚的基础。





雨课堂 Rain Classroom



# Thank you!